

Bloc 2 TP Firebase / Supabase

Contexte	2
Configuration de Firebase	2
Application Todolist	19
Connexion / Inscription	32
Photo de profil (Firebase Storage)	44
Passage à Supabase	44
Exercices Supabase	63
Comparatif Firebase / Supabase	80
Conclusion	80

Contexte

Ce TP va nous faire découvrir **Firestore**, le backend de Google, et l'utiliser dans des applications **Flutter**. Il présentera la configuration d'un projet, une application **Todolist** avec **Cloud Firestore**, une interface de connexion / inscription avec **Firestore Authentication** et le stockage d'une photo de profil.

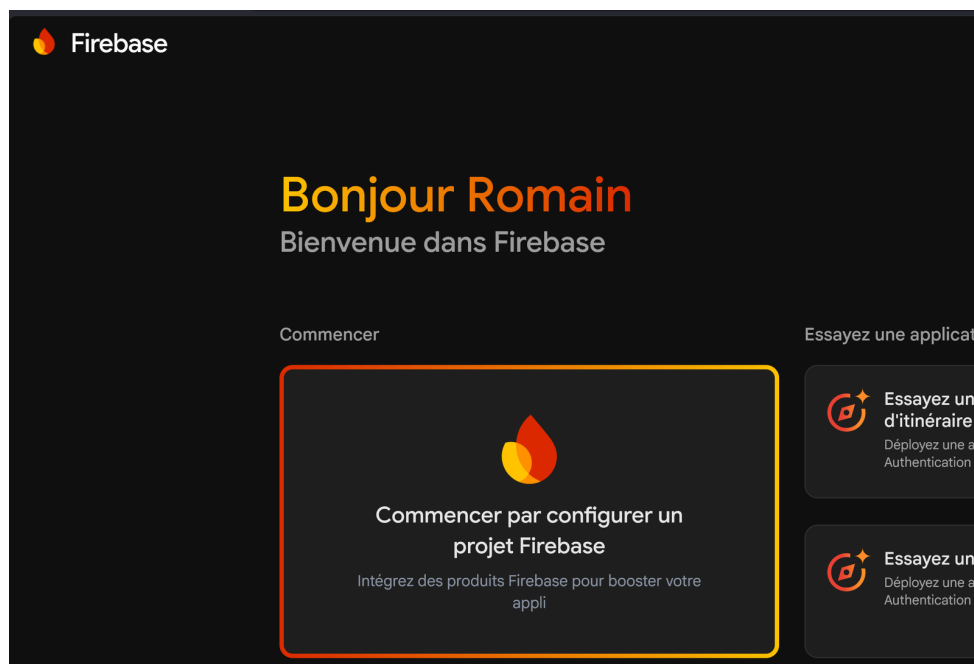
La partie photo de profil utilise **Firestore Storage** qui n'est plus gratuit, on l'a donc refaite avec **Supabase** (une alternative open source basé sur **PostgreSQL**) et on a fait les exercices du cours Supabase à la fin.

Tout est testé sur un émulateur **Android** (Pixel 7) car on a pas de Mac pour la partie iOS.

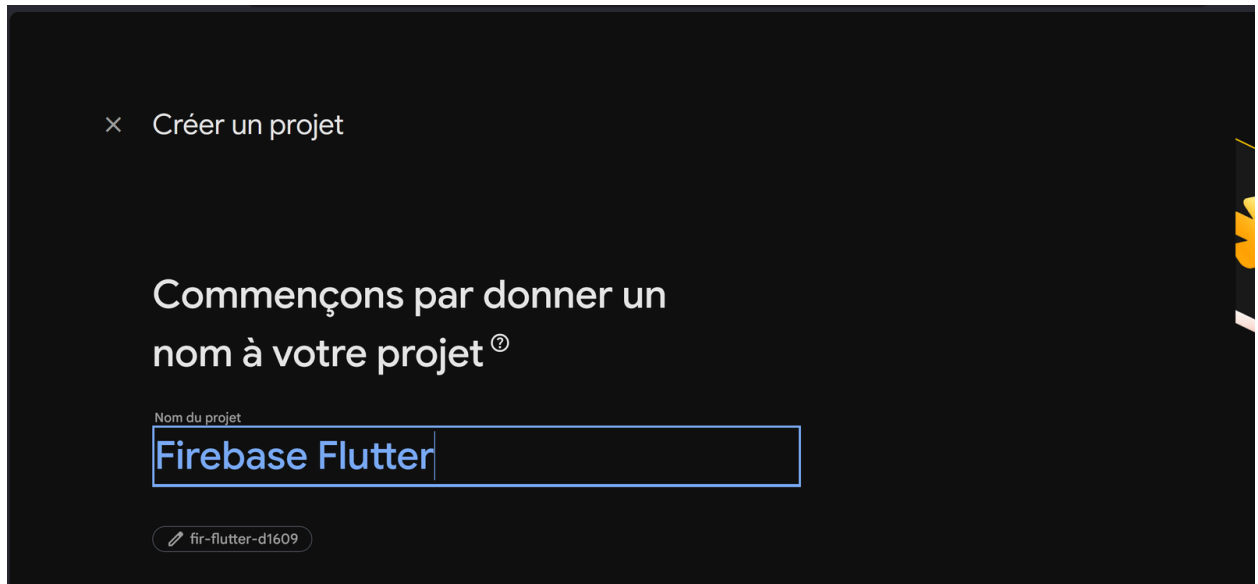
(Les projets seront disponible sur GitHub accessible via le portfolio).

Configuration de Firestore

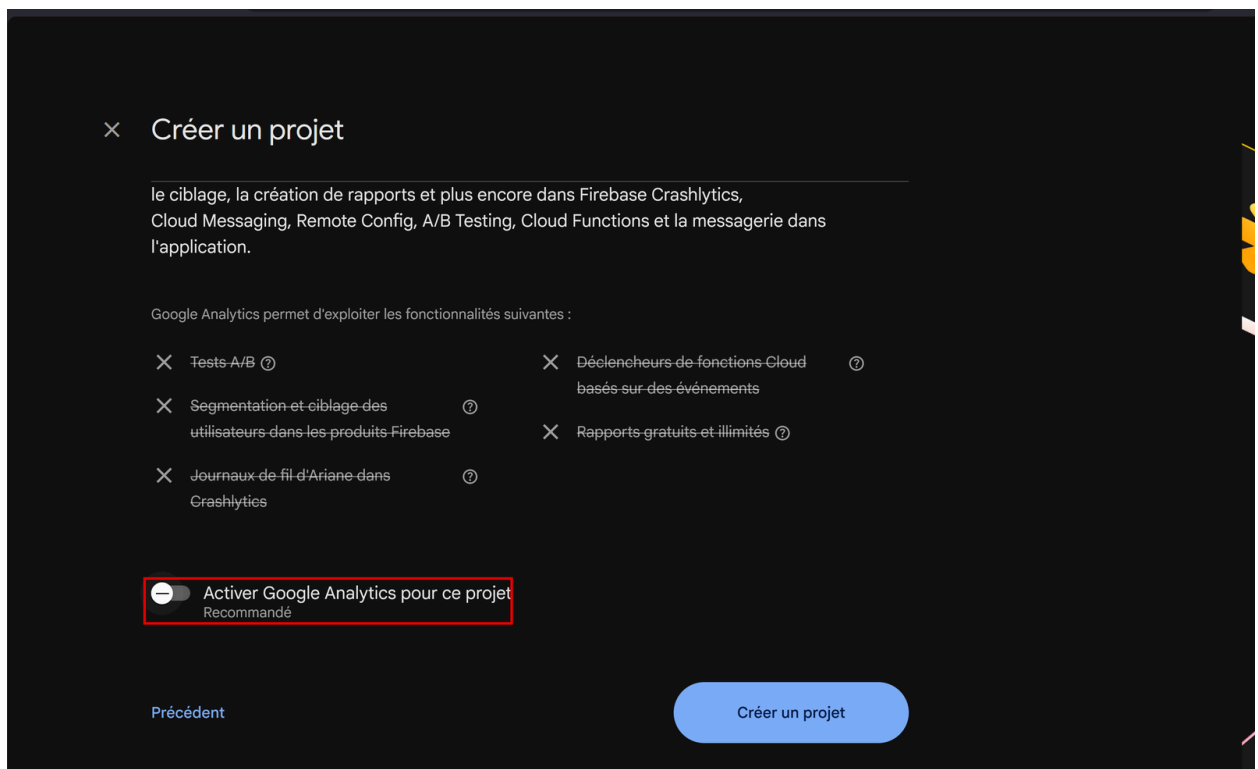
On se rend sur la console Firestore (<https://console.firebase.google.com>) avec notre compte Google et on clique sur “Commencer par configurer un projet Firestore” :



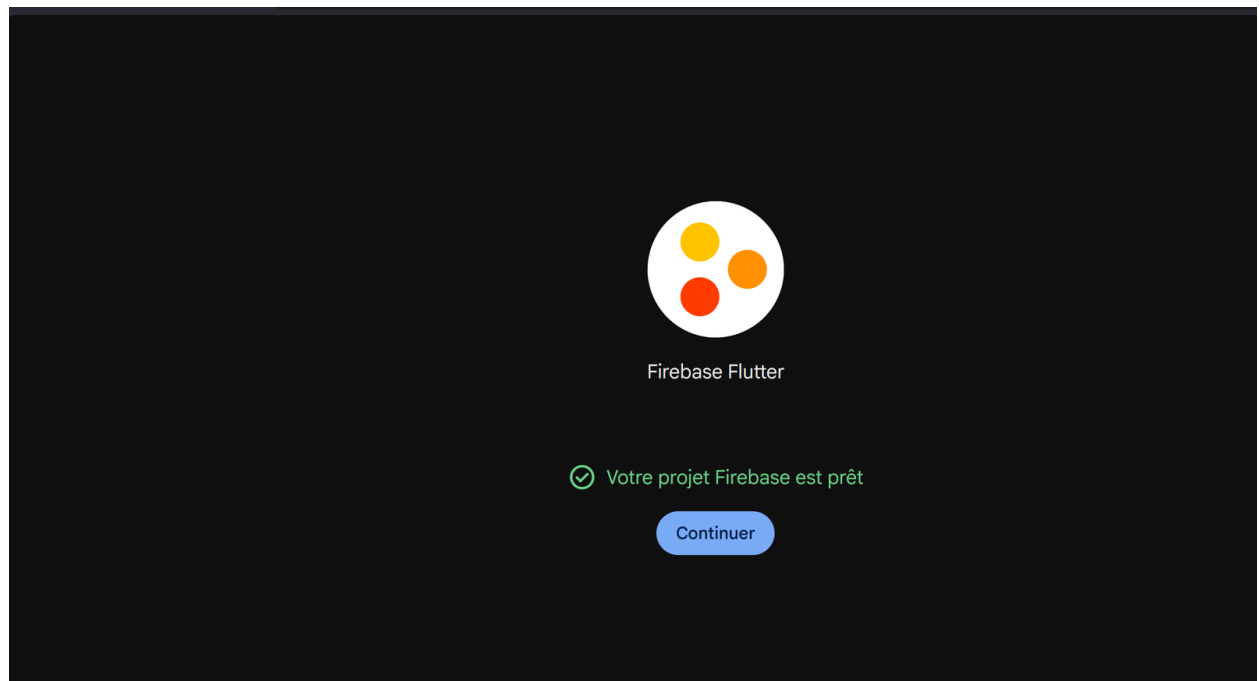
On donne un nom au projet, ici “**Firebase Flutter**” :



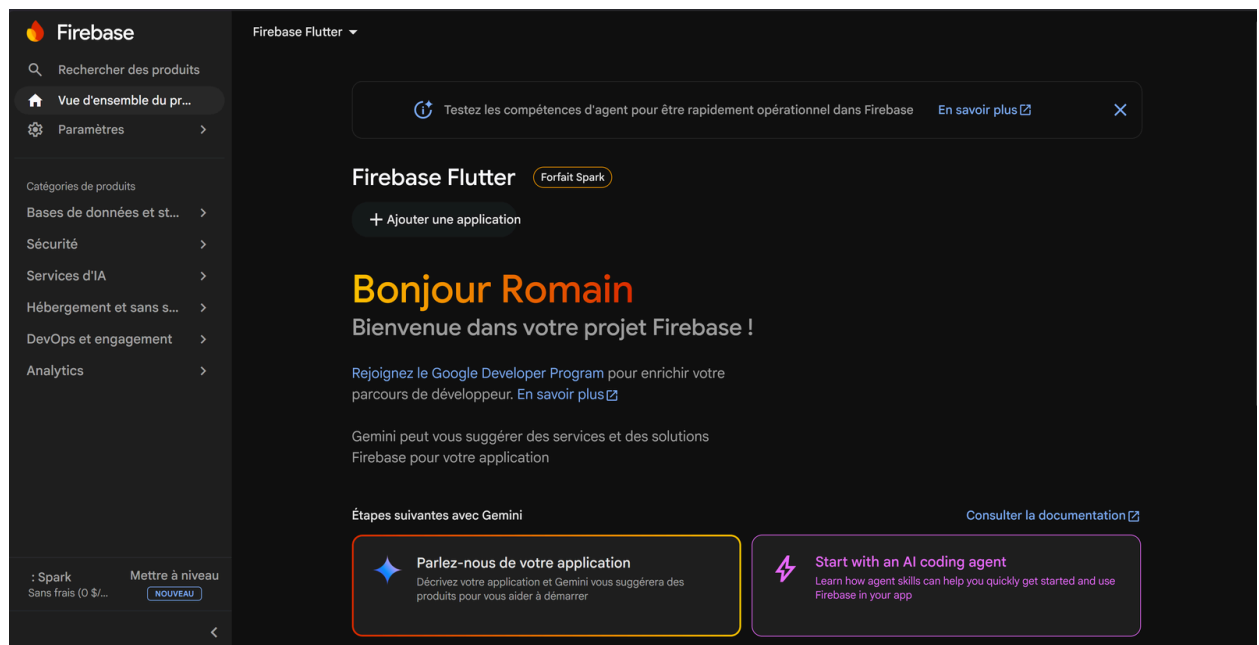
Comme indiqué dans le TP on n’active pas **Google Analytics**, c’est un outil plus avancé qu’on utilisera pas ici :



Le projet est créé, on clique sur “**Continuer**” :

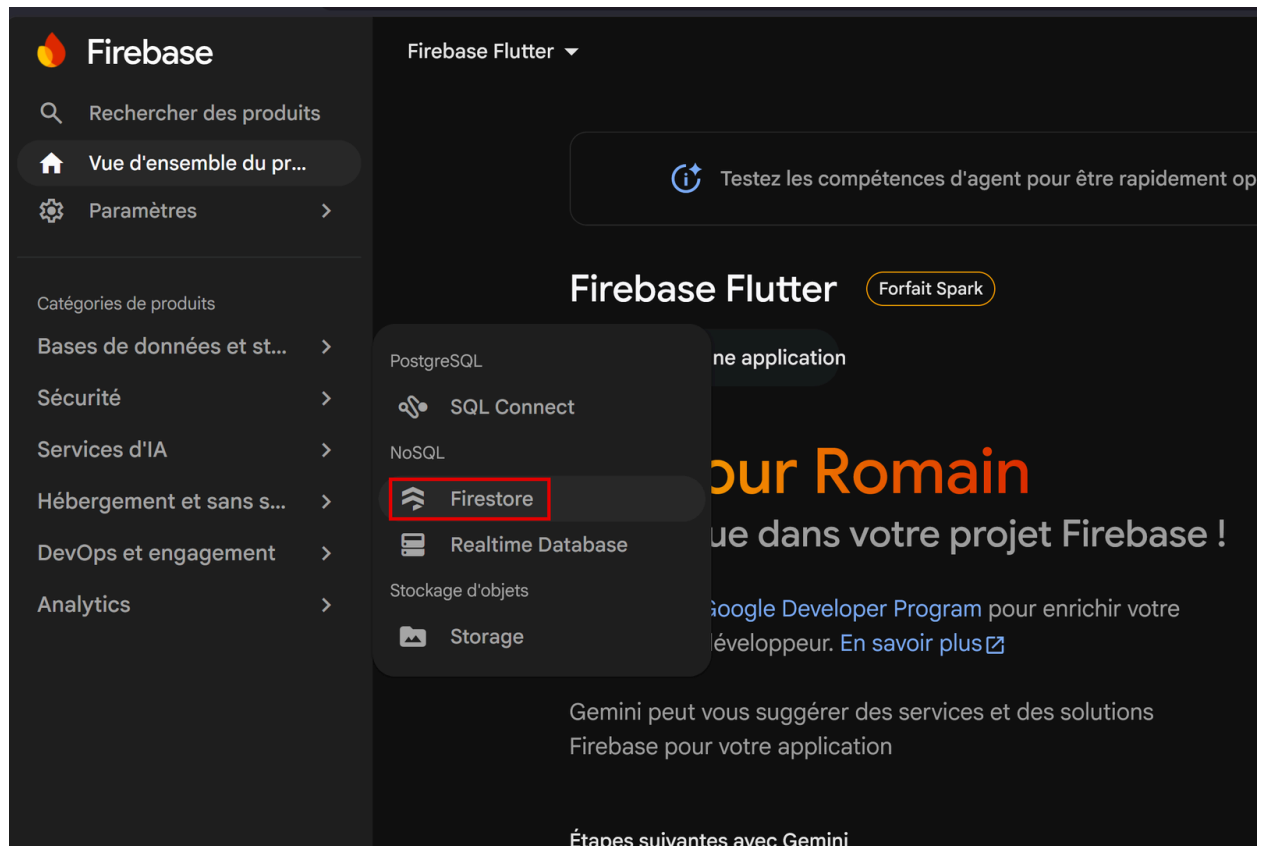


On arrive sur le tableau de bord du projet, qui donne accès à tous les services de Firebase :

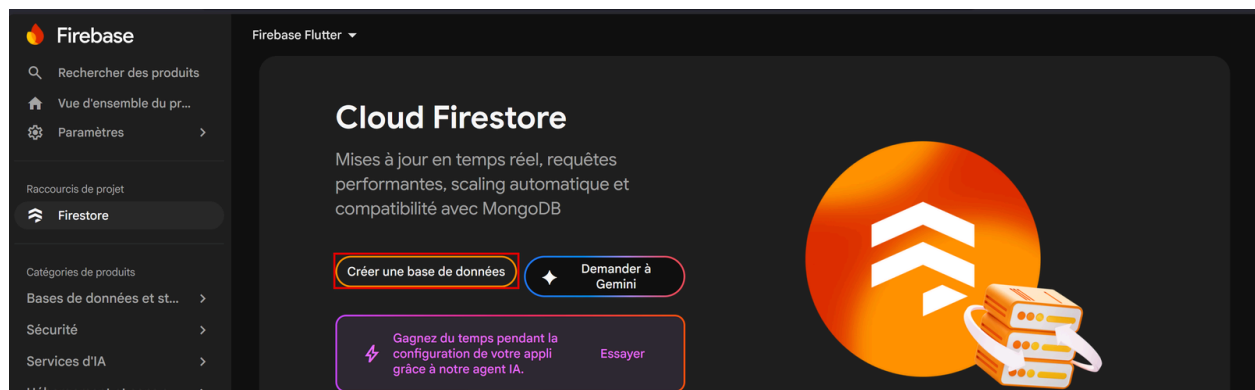


Création de la base Firestore

Pour tester la connexion entre Flutter et Firebase on crée une première base **Firestore**. Le menu a changé depuis le TP, Firestore se trouve maintenant dans “Bases de données et stockage” :



On clique sur “Créer une base de données” :



On laisse l'édition **Standard** :

× Créer une base de données

1 Sélectionner l'édition

☒ Édition Standard
Moteur de requêtes simple avec indexation automatique
Compatible avec les opérations essentielles

☐ Édition Enterprise
Moteur de requêtes avancé avec indexation autogérée.
Compatible avec les opérations essentielles, les opérations de pipelines et les opérations MongoDB

Vous ne savez pas quelle édition choisir ? [Comparez les éditions](#) 

Suivant

On choisie l'emplacement **eur3 (Europe)** :

× Créer une base de données

Sélectionner l'édition

2 ID et emplacement de la base de données

ID de la base de données

(default)

Emplacement

eur3 (Europe)

ⓘ Le paramètre d'emplacement définit l'endroit où sont stockées vos données Cloud Firestore

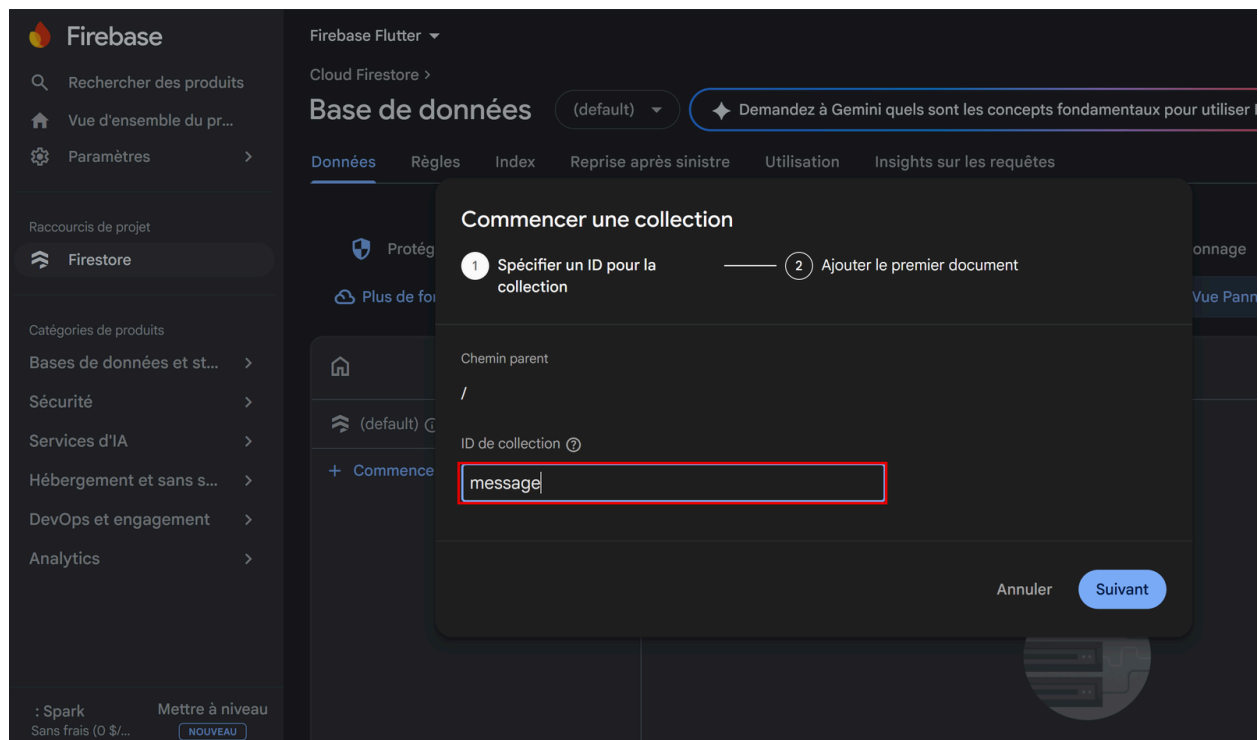
! Une fois défini, cet emplacement ne pourra plus être modifié

En savoir pl

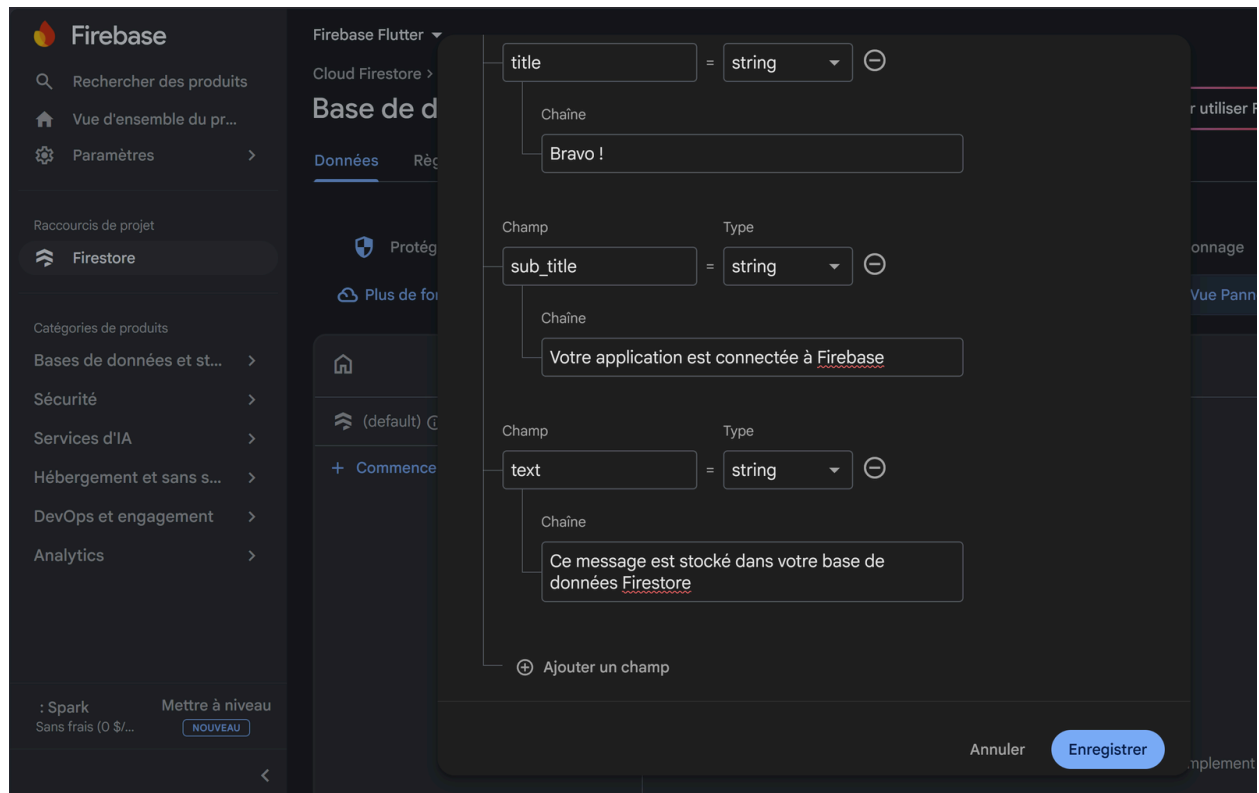
On démarre en **mode test** pour pouvoir lire et écrire dans la base sans règles de sécurité (attention c'est valable seulement 30 jours) :



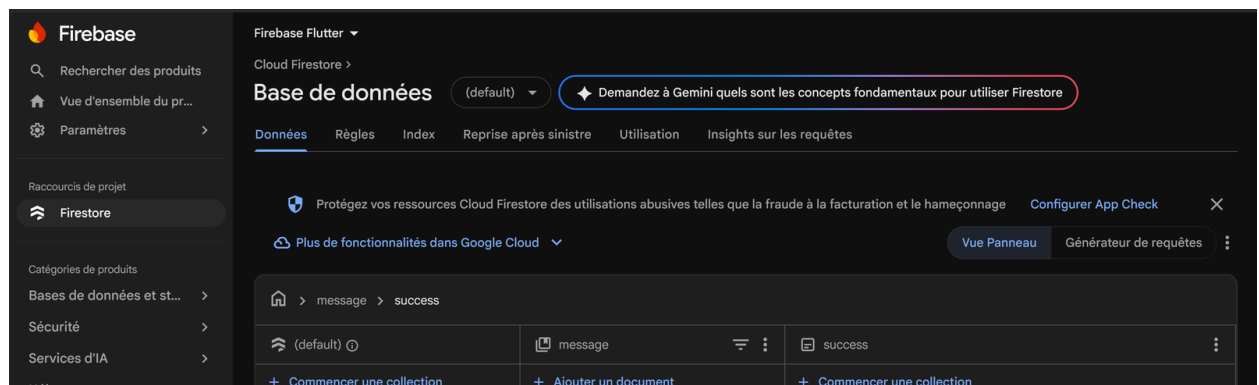
La base est vide, on clique sur “**Commencer une collection**” et on la nomme “**message**” :



On crée le document “**success**” avec trois champs de type **string** : *title*, *sub_title* et *text* :



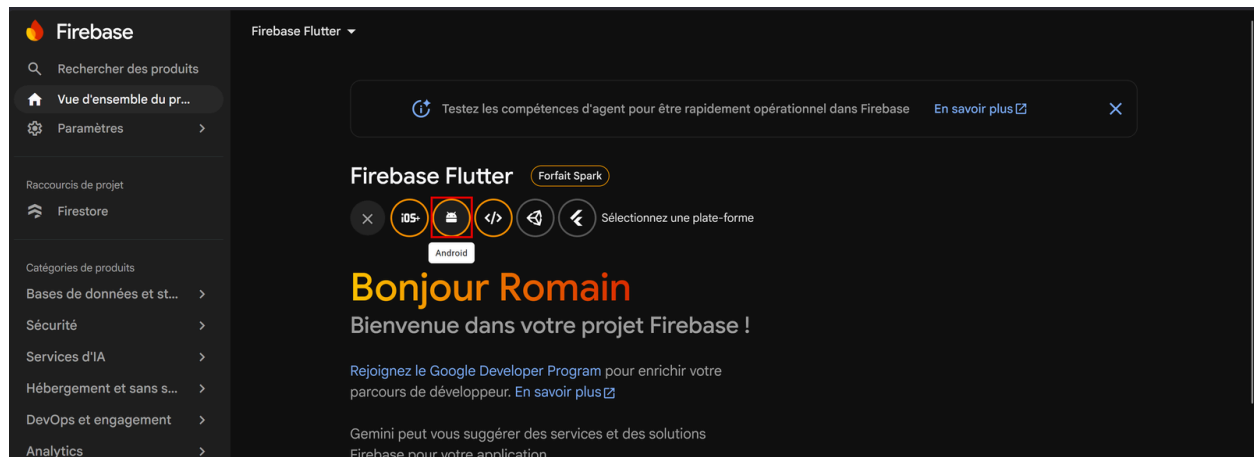
Voilà le rendu final de la collection **message** :



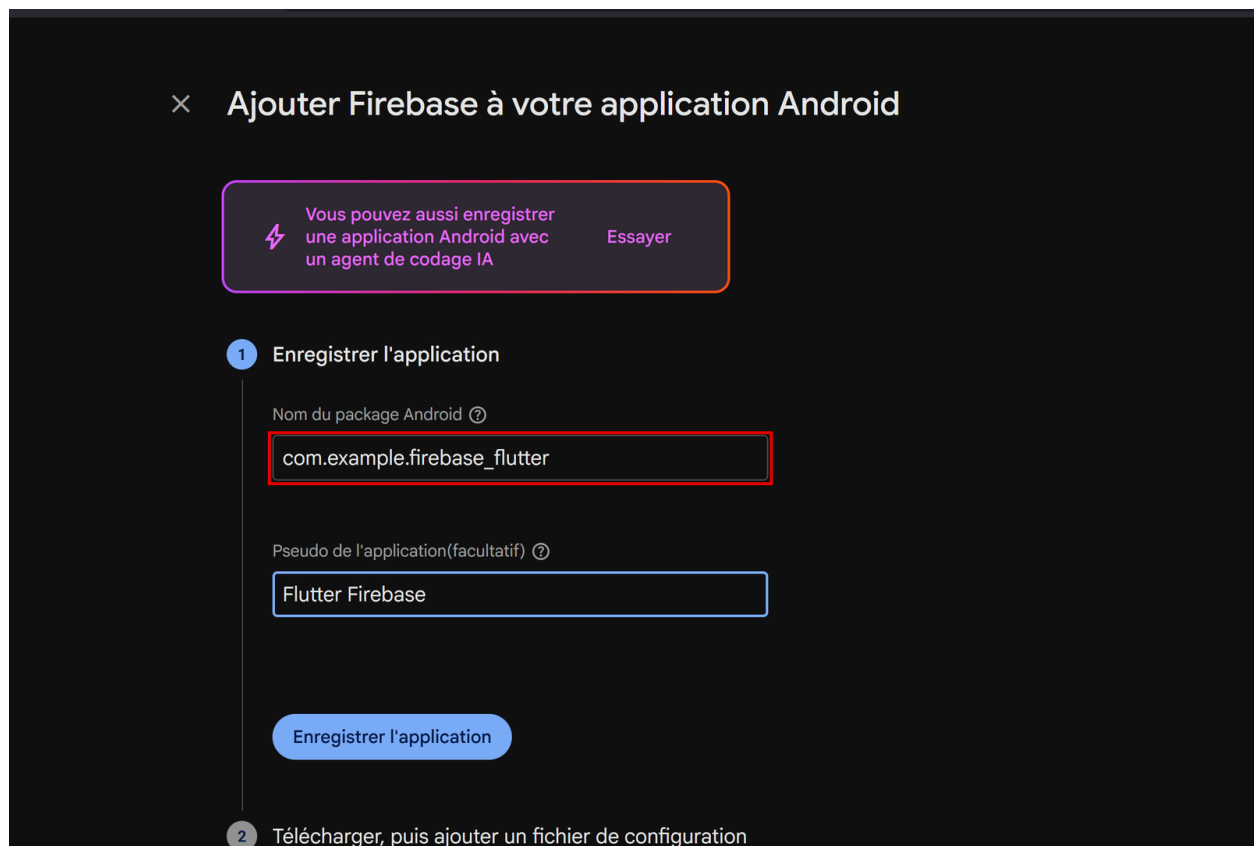
Configuration Android

On n’a pas de Mac donc on ne peut pas faire la partie **iOS** (Xcode existe seulement sur macOS), on passe directement à Android.

On crée le projet Flutter avec `flutter create firebase_flutter`, puis dans la console on ajoute une application **Android** :



Le nom du package se trouve dans `android/app/build.gradle.kts` (`applicationId`), ici `com.example.firebase_flutter`. On laisse le **SHA-1** vide :



On télécharge le fichier **google-services.json** et on le place dans le dossier *android/app/* du projet :

×

Ajouter Firebase à votre application Android

⚡

Vous pouvez aussi enregistrer une application Android avec un agent de codage IA

Essayer

✓

Enregistrer l'application

Nom du package Android : com.example.firebase_flutter, pseudo de l'application : Flutter Firebase

2

Télécharger, puis ajouter un fichier de configuration

pour Android Studio ci-dessous | [Unity](#) [C++](#)

↓

Télécharger google-services.json

Accédez à la vue Projet dans Android Studio pour consulter le répertoire racine de votre projet.

Project

MyApplication [My Application]

gradle

idea

app

libs

src

.gitignore

build.gradle.kts

google-services.json

proguard-rules.pro

gradle

Déplacez le fichier `google-services.json` que vous avez téléchargé dans le répertoire racine de votre module (au niveau de l'application).

Firebase nous indique ensuite les modifications à faire dans les fichiers **Gradle**. Les projets Flutter récent utilisent le **Kotlin DSL** (.kts) et plus le *build.gradle* du TP, la syntaxe est un peu différente :

3 Ajouter le SDK Firebase

Instructions pour Gradle | [Unity](#) [C++](#)

★ Vous utilisez toujours la syntaxe buildscript pour gérer les plug-ins ? Découvrez comment ajouter des plug-ins Firebase à l'aide de cette syntaxe.

1. Pour rendre les valeurs de configuration google-services.json accessibles aux SDK Firebase, vous devez disposer du plug-in Gradle des services Google.

☒ DSL Kotlin (build.gradle.kts)

☐ Groovy (build.gradle)

Ajoutez le plug-in en tant que dépendance du fichier build.gradle.kts au niveau du projet :

Fichier Gradle au niveau racine (au niveau du projet) (<project>/build.gradle.kts):

```
plugins {  
    // ...  
  
    // Add the dependency for the Google services Gradle plugin  
    id("com.google.gms.google-services") version "4.5.0" apply false  
}
```

2. Ensuite, dans le fichier build.gradle.kts de votre module (au niveau de l'application), ajoutez le plug-in google-services et tous les SDK Firebase que vous souhaitez utiliser dans votre application :

Fichier Gradle du module (au niveau de l'application) (<project>/<app-module>/build.gradle.kts):

```
plugins {  
    id("com.android.application")  
    // Add the Google services Gradle plugin
```

12

On ajoute donc le plugin **google-services** dans *android/settings.gradle.kts* :

```
pluginManagement {
    val flutterSdkPath =
        run {
            val properties = java.util.Properties()
            file("local.properties").inputStream().use { properties.load(it) }
            val flutterSdkPath = properties.getProperty("flutter.sdk")
            require(flutterSdkPath != null) { "flutter.sdk not set in local.properties" }
            flutterSdkPath
        }

    includeBuild("$flutterSdkPath/packages/flutter_tools/gradle")

    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

plugins {
    id("dev.flutter.flutter-plugin-loader") version "1.0.0"
    id("com.android.application") version "9.1.0" apply false
    id("org.jetbrains.kotlin.android") version "2.4.0" apply false
    // Add the dependency for the Google services Gradle plugin
    id("com.google.gms.google-services") version "4.5.0" apply false
}

include(":app")
```

Puis on l'applique dans *android/app/build.gradle.kts* :

```
plugins {
    id("com.android.application")
    // Add the Google services Gradle plugin
    id("com.google.gms.google-services")
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
    id("dev.flutter.flutter-gradle-plugin")
}

android {
    namespace = "com.example.firebase_flutter"
    compileSdk = flutter.compileSdkVersion
    ndkVersion = flutter.ndkVersion

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    defaultConfig {
        // TODO: Specify your own unique Application ID (https://developer.android.com/studio/build/application-id.html)
        applicationId = "com.example.firebase_flutter"
        // You can update the following values to match your application needs.
        // For more information, see: https://flutter.dev/to/review-gradle-config.
        minSdk = flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        // Uses the version code from pubspec.yaml. When using split APKs, 1000 * ABI_VERSION
        // is added automatically by Flutter. (https://developer.android.com/studio/build/configure-apk-splits#configure)
        // You can force using the value of versionCode by specifying the `-P force-version-code-ignoring-abi=true`
        // flag during build.
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }
}
```

Le TP dit aussi de mettre **minSdkVersion 21**, mais Flutter met déjà **24** par défaut (*flutter.minSdkVersion*) donc pas besoin de le changer.

Tester la connexion

On ajoute les packages **firebase_core** et **cloud_firestore** avec **flutter pub add**, les versions sont beaucoup plus récente que dans le TP :

```
environment:
  sdk: ^3.13.3

# Dependencies specify other packages that your package needs in order to work.
# To automatically upgrade your package dependencies to the latest versions
# consider running `flutter pub upgrade --major-versions`. Alternatively,
# dependencies can be manually updated by changing the version numbers below to
# the latest version available on pub.dev. To see which dependencies have newer
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  firebase_core: ^4.15.0
  cloud_firestore: ^6.10.0

dev_dependencies:
  flutter_test:
    sdk: flutter

  # The "flutter_lints" package below contains a set of recommended lints to
  # encourage good coding practices. The lint set provided by the package is
  # activated in the `analysis_options.yaml` file located at the root of your
  # package. See that file for information about deactivating specific lint
  # rules and activating additional ones.
  flutter_lints: ^6.0.0
```

Le **main()** initialise Firebase avant de lancer l'application :

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

Run | Debug | Profile
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Firebase',
      home: HomePage(),
    ); // MaterialApp
  }
}

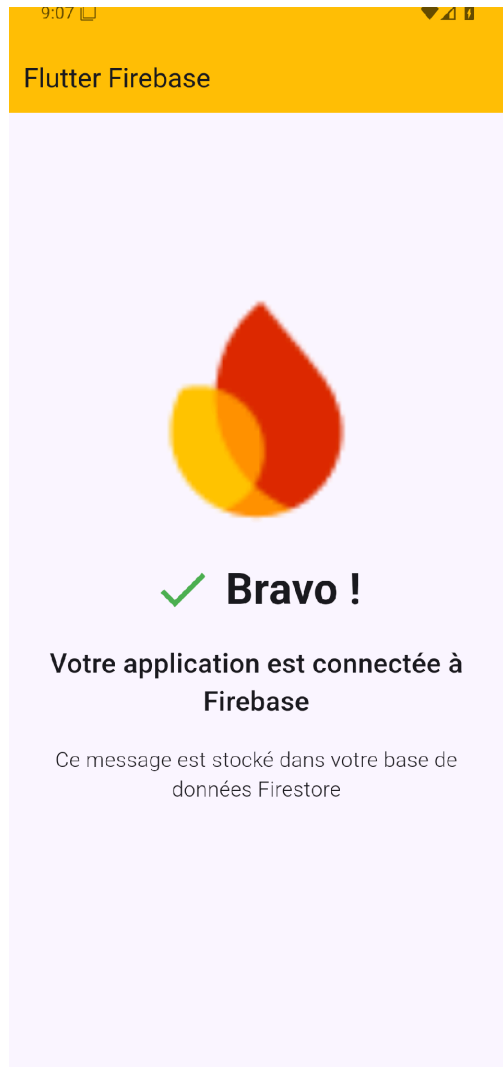
class HomePage extends StatefulWidget {
  const HomePage({Key? key}) : super(key: key);
  @override
  _HomePageState createState() {
    return _HomePageState();
  }
}
```

Le widget **FirebaseMessage** récupère le document **success** de la collection **message** avec un **FutureBuilder** et l'envoie au widget **MessageDesign** :

```
}  
  
class FirebaseMessage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    CollectionReference users =  
      FirebaseFirestore.instance.collection('message');  
    return FutureBuilder<DocumentSnapshot>(  
      future: users.doc('success').get(),  
      builder:  
        (BuildContext context, AsyncSnapshot<DocumentSnapshot> snapshot) {  
          if (snapshot.hasError) {  
            return Text("Quelque chose s'est mal passé");  
          }  
          if (snapshot.hasData && !snapshot.data!.exists) {  
            return Text("Le document n'existe pas");  
          }  
          if (snapshot.connectionState == ConnectionState.done) {  
            Map<String, dynamic> data =  
              snapshot.data!.data() as Map<String, dynamic>;  
            return MessageDesign(  
              titleMessage: data["title"],  
              subtitleMessage: data["sub_title"],  
              textMessage: data["text"],  
            );  
          }  
          return Text("Chargement en cours");  
        },  
    );  
  }  
}
```

Au premier lancement, **Gradle** n'arrivait pas à télécharger ses dépendances à cause du **proxy** du lycée, il a fallu l'ajouter dans le fichier *gradle.properties* de l'utilisateur.

On lance l'application sur l'émulateur, le message stocké dans Firestore s'affiche bien donc la configuration fonctionne :



Application Todolist

On crée un nouveau projet **todolist** et on ajoute **firebase_core**, **cloud_firestore** et **intl**. On enregistre aussi cette application Android dans la console comme dans la première partie.

1. Envoyer des données dans Firestore

La classe **InitializeApp** utilise un **FutureBuilder** pour attendre **Firestore.initializeApp()** avant d'afficher la page, avec un widget d'erreur et un de chargement :

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

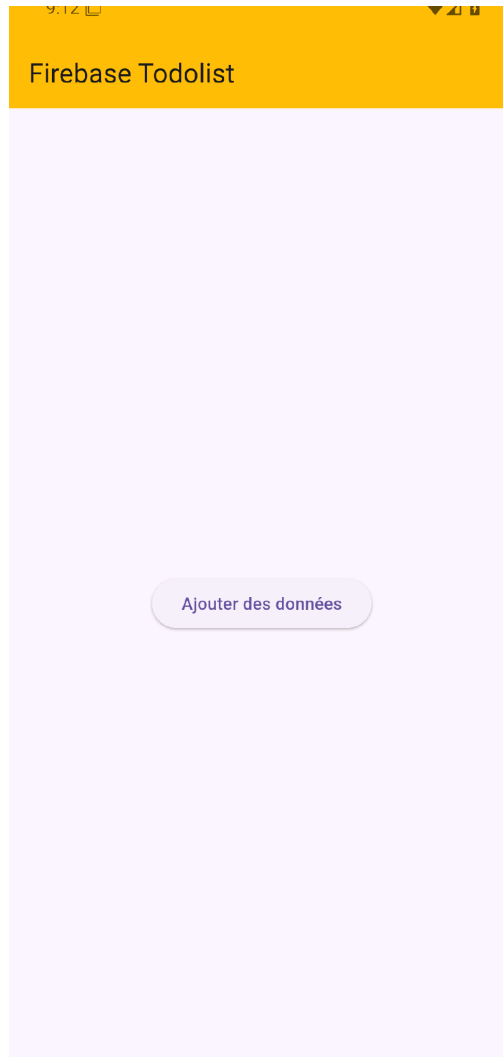
Run | Debug | Profile
void main() => runApp(const InitializeApp());

class InitializeApp extends StatelessWidget {
  const InitializeApp({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return FutureBuilder<
      future: Firestore.initializeApp(),
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return const ErrorFirestore();
        }
        if (snapshot.connectionState == ConnectionState.done) {
          return MyApp();
        }
        return const Loading();
      },
    ); // FutureBuilder
  }
}
```

On commence par un simple bouton qui ajoute toujours le même document dans la collection **"items"** avec la fonction **addDataToFirebase()** :

```
class MyApp extends StatelessWidget {  
  MyApp({Key? key}) : super(key: key);  
  final databaseReference = FirebaseFirestore.instance;  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      theme: ThemeData(  
        primarySwatch: Colors.amber,  
      ), // ThemeData  
      home: Scaffold(  
        appBar: AppBar(  
          title: const Text('Firebase Todolist'),  
          backgroundColor: Colors.amber,  
        ), // AppBar  
        body: Center(  
          child: ElevatedButton(  
            onPressed: () {  
              addDataToFirebase();  
            },  
            child: const Text('Ajouter des données'),  
          ), // ElevatedButton  
        ), // Center  
      ), // Scaffold  
    ); // MaterialApp  
  }  
  
  void addDataToFirebase() {  
    try {  
      databaseReference.collection("items").add({  
        "text": "Lire un livre pendant 30 minutes",  
      }).then((value) {  
        print(value.id);  
      });  
    } catch (e) {  
      print(e.toString());  
    }  
  }  
}
```

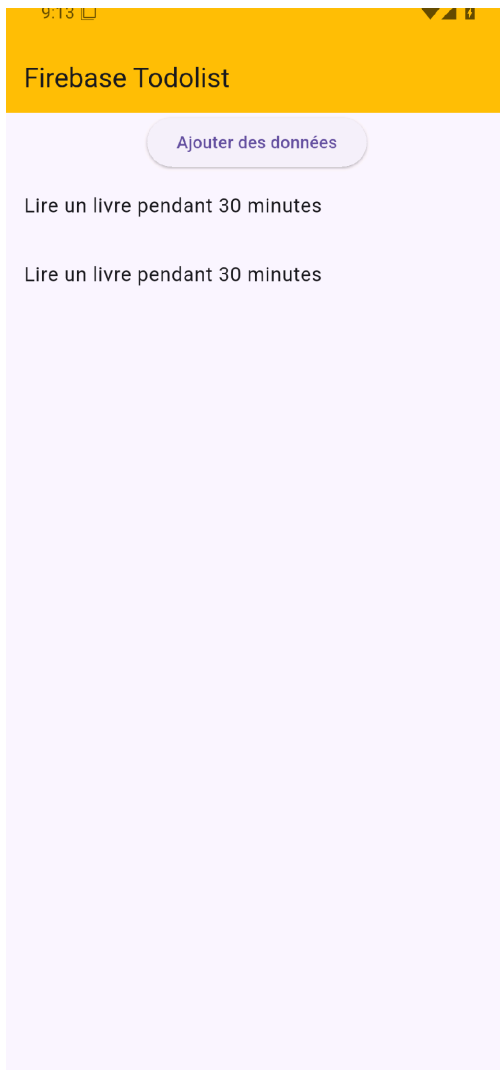
Rendu sur l'émulateur, à chaque clique l'identifiant du nouveau document s'affiche dans la console (ici *6vF2XkpEHHRsGJDygTxT* puis *laNmTCMvPeyuLmSNhsJu*) :



2. Récupérer les données de Firestore

On ajoute ensuite la **ListSection** qui utilise un **StreamBuilder** sur la collection **items**, la liste se met à jour automatiquement à chaque ajout :

```
class ListSection extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return StreamBuilder(  
      stream: databaseReference.collection('items').snapshots(),  
      builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> snapshot) {  
        if (!snapshot.hasData) return const Text('Loading...');  
        return ListView(  
          children: snapshot.data!.docs.map((document) {  
            return ListTile(  
              title: Text(document['text']),  
            ); // ListTile  
          }).toList(),  
        ); // ListView  
      },  
    ); // StreamBuilder  
  }  
}
```



3. Formulaire d'envoi de données

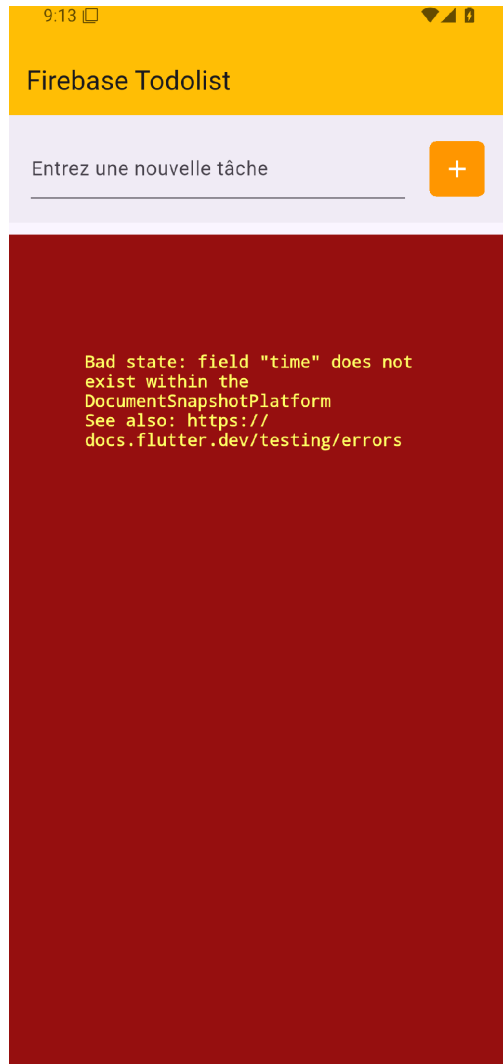
On remplace le bouton par un formulaire (**FormSection**) avec un **TextField** et un bouton ajouter. On enregistre aussi l'heure avec le package **intl** et le champ booléen **"done"** :

```
class FormSection extends StatelessWidget {
  void addItem() {
    try {
      var now = DateTime.now();
      var hourAndMinutes = DateFormat('HH:mm');
      databaseReference.collection("items").add({
        "text": myController.text,
        "time": hourAndMinutes.format(now),
        "done": false,
      }).then((value) {
        print(value.id);
        myController.clear();
      });
    } catch (e) {
      print(e.toString());
    }
  }
}

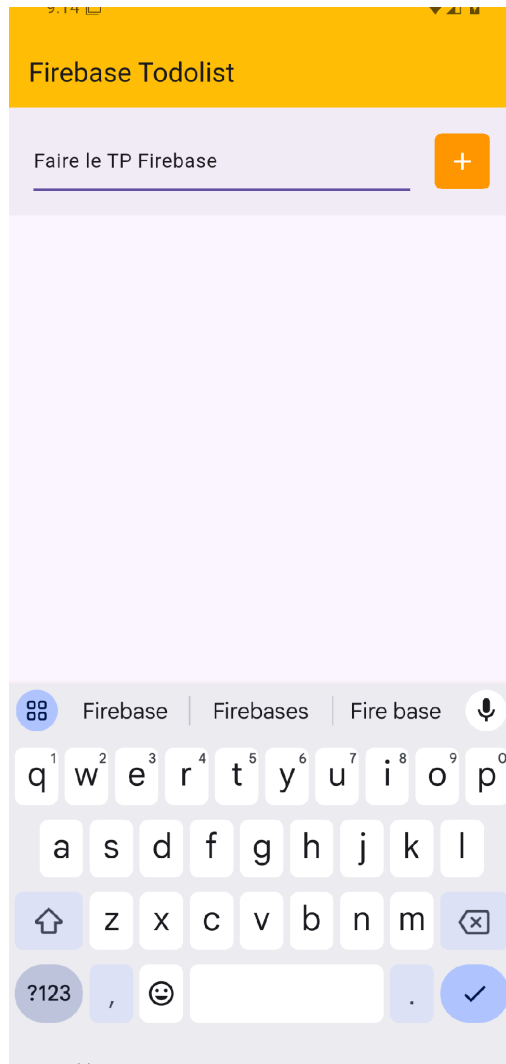
class ListSection extends StatelessWidget {
  ListSection({Key? key}) : super(key: key);
  final databaseReference = FirebaseFirestore.instance;
  @override
  Widget build(BuildContext context) {
    return StreamBuilder(
      stream: databaseReference.collection('items').snapshots(),
      builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> snapshot) {
        if (!snapshot.hasData) return const Text('Loading...');
        return ListView(
          children: snapshot.data!.docs.map((document) {

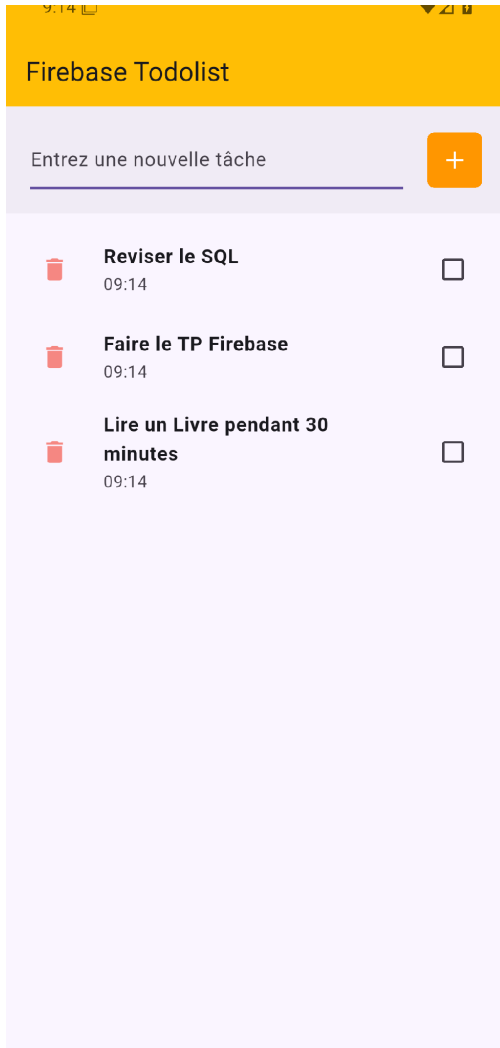
```

Si on relance l'application sans vider la collection on a cette erreur, car les anciens documents n'ont pas de champ **time**. C'est pour ça que le TP dit de vider la collection **items** avant :



Après avoir supprimé les anciens documents, on peut ajouter des tâches :





4. Mettre à jour les données

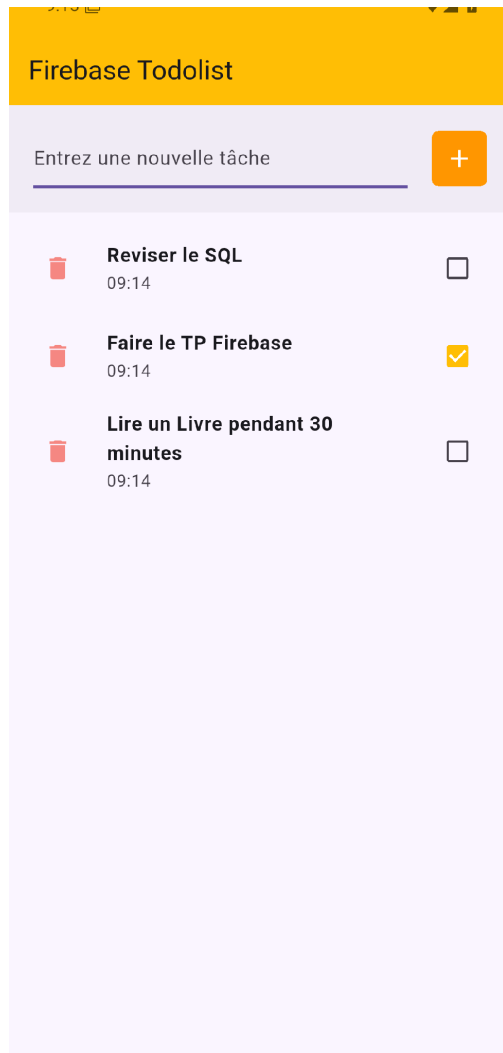
La **ListSection** utilise maintenant des **CheckboxListTile**, la case à cocher appelle **update()** pour modifier le champ **done** et la corbeille appelle **delete()** :

```
@override
Widget build(BuildContext context) {
  return StreamBuilder(
    stream: databaseReference.collection('items').snapshots(),
    builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> snapshot) {
      if (!snapshot.hasData) return const Text('Loading...');
      return ListView(
        children: snapshot.data!.docs.map((document) {
          return CheckboxListTile(
            title: Text(
              document['text'],
              style: const TextStyle(fontWeight: FontWeight.bold),
            ), // Text
            subtitle: Text(document['time']),
            value: document['done'],
            activeColor: Colors.amber,
            secondary: IconButton(
              icon: Icon(
                Icons.delete,
                color: Colors.red.withOpacity(0.6),
              ), // Icon
              onPressed: () {
                deleteItem(document.id);
              },
            ), // IconButton
            onChanged: (bool? value) {
              print(value);
              updateItem(document.id, value!);
            },
          ); // CheckboxListTile
        }).toList(),
      ); // ListView
    },
  ); // StreamBuilder
}

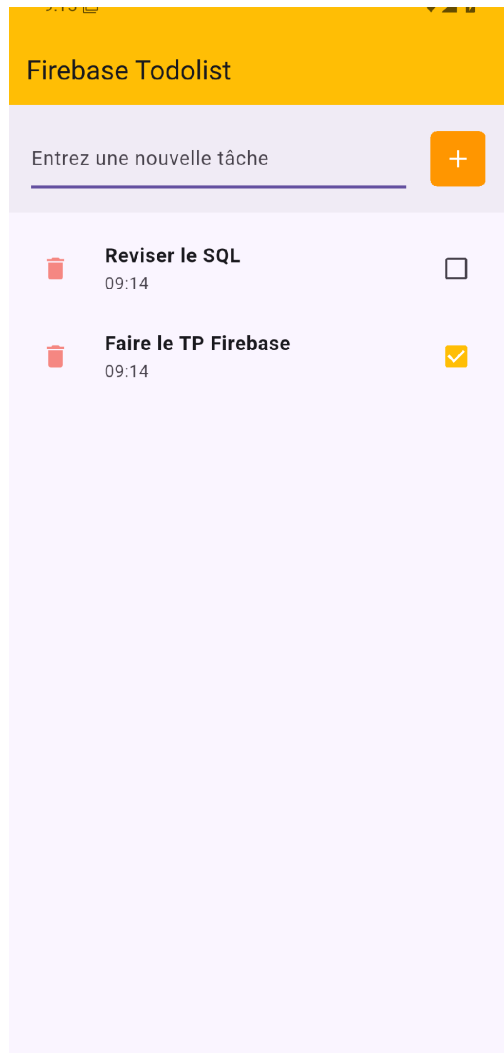
void deleteItem(String itemID) {
  databaseReference.collection("items").doc(itemID).delete();
}

void updateItem(String itemID, bool itemDone) {
  databaseReference.collection("items").doc(itemID).update({
    'done': itemDone,
  });
}
```

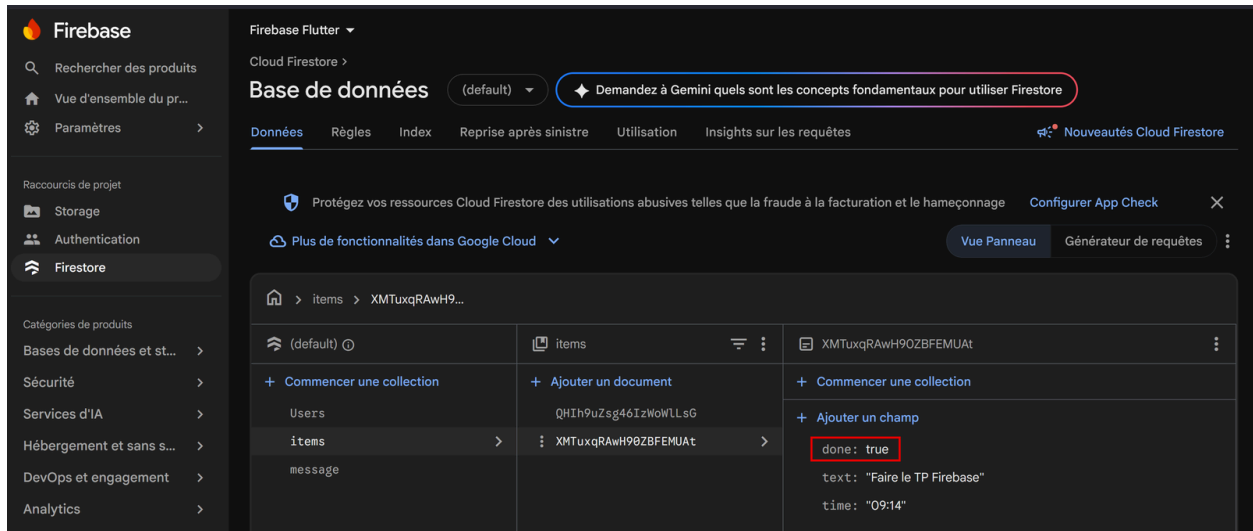
On coche une tâche, le champ **done** passe à **true** dans Firestore :



Et on en supprime une :



Dans la console Firestore il reste bien les 2 tâches, et celle qu'on a cochée a le champ **done** à **true** :

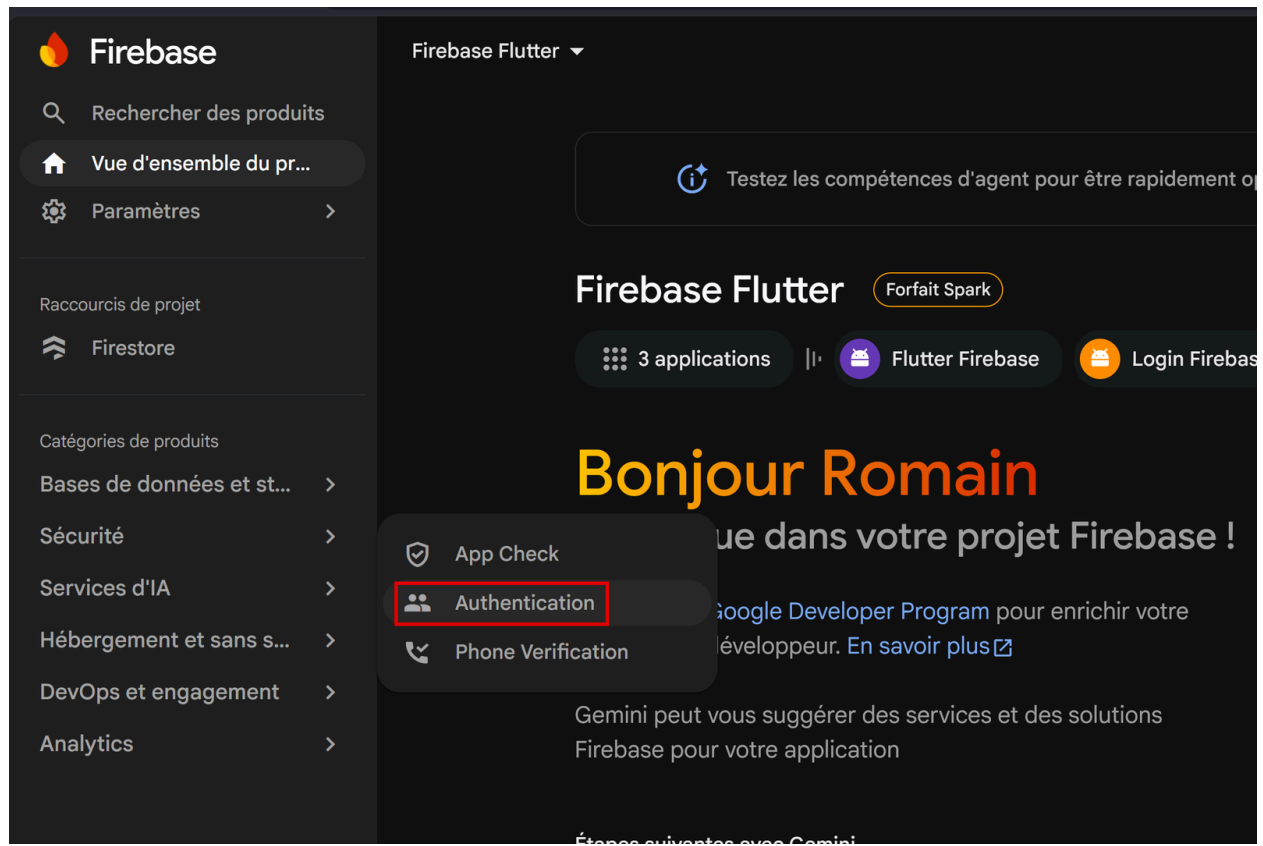


The screenshot shows the Firebase console interface. On the left is a sidebar with navigation options like 'Rechercher des produits', 'Vue d'ensemble du pr...', 'Paramètres', and 'Raccourcis de projet'. The main area is titled 'Base de données' (Database) and shows the 'items' collection. A document with ID 'XMTuxqRAwH90ZBFEMUAt' is selected, and its fields are visible: 'done' (true), 'text' ('Faire le TP Firebase'), and 'time' ('09:14'). The 'done' field is highlighted with a red box.

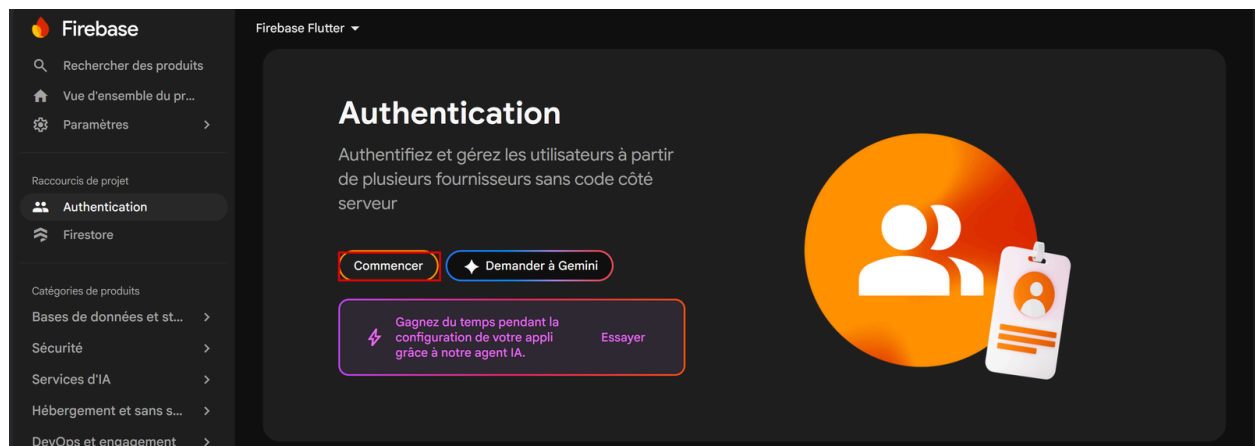
Collection	Document ID	Field	Value
(default) items	XMTuxqRAwH90ZBFEMUAt	done	true
		text	"Faire le TP Firebase"
		time	"09:14"

Connexion / Inscription

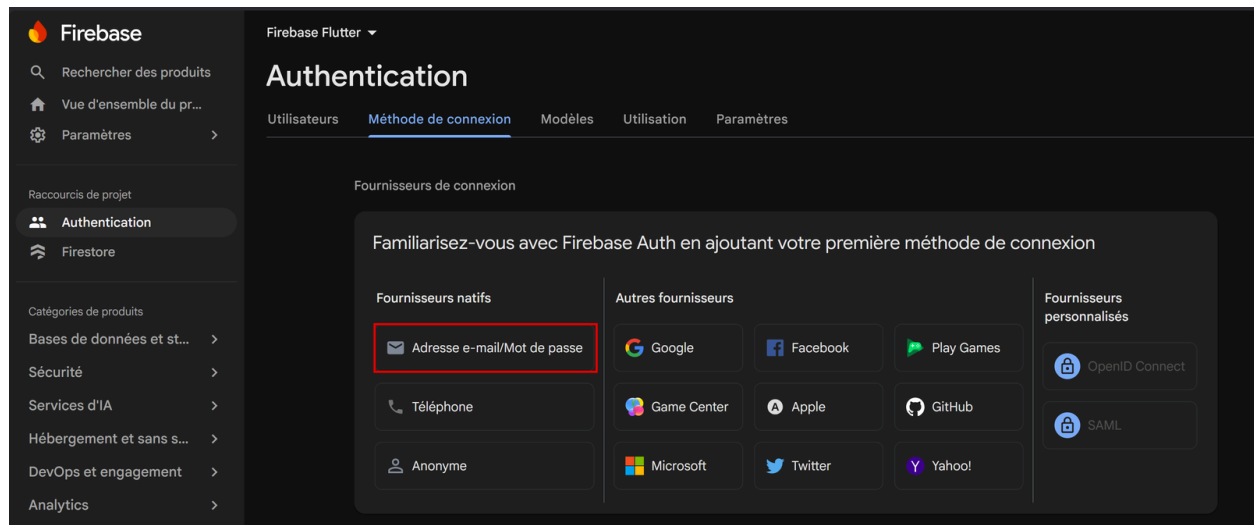
On crée le projet **login_firebase** avec le package **firebase_auth** en plus. Dans la console on doit activer l'authentification, le menu **Authentication** se trouve maintenant dans “Sécurité” :



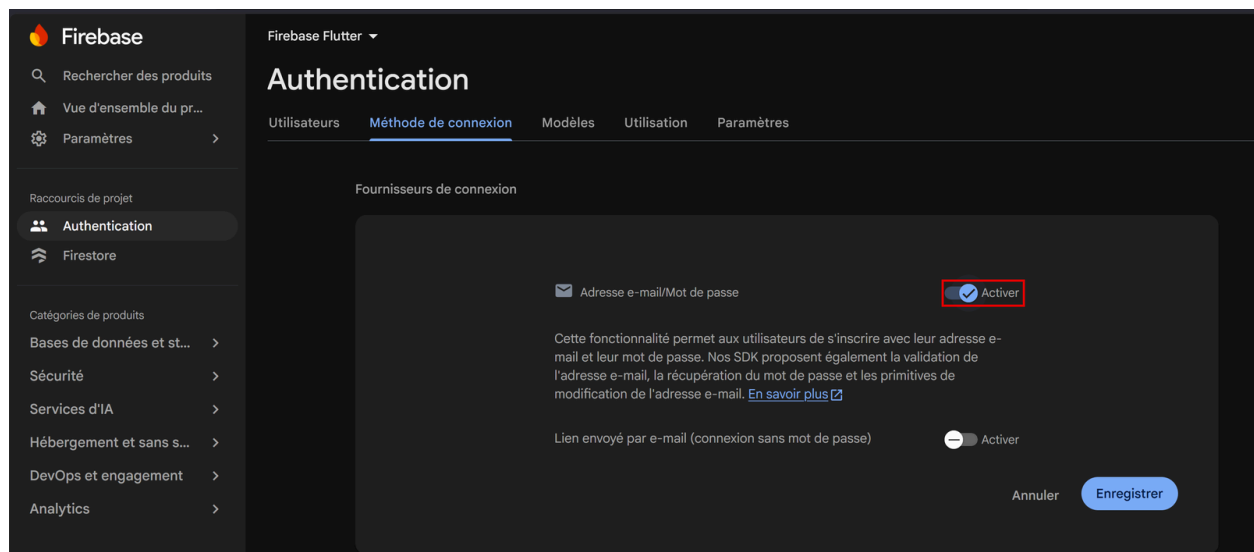
On clique sur “Commencer” :



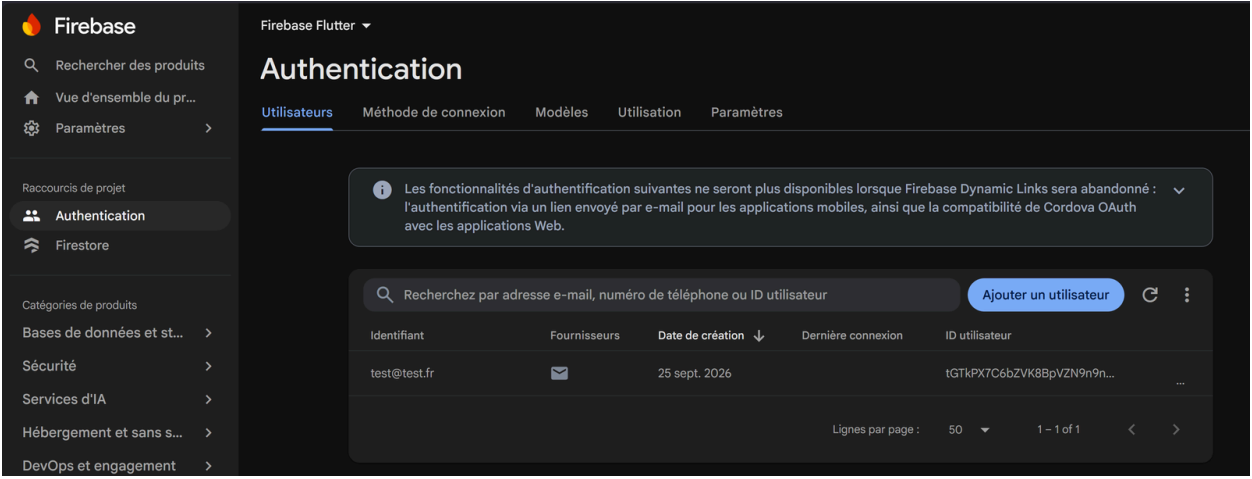
On choisie le mode de connexion **Adresse e-mail/Mot de passe** :



On l'active puis on clique sur “Enregistrer” :



Dans l'onglet **Utilisateurs** on ajoute un utilisateur de test (**test@test.fr**), il apparaît avec son identifiant Firebase :



The screenshot shows the Firebase Authentication console for a Flutter project. The left sidebar contains the Firebase logo and navigation links: 'Rechercher des produits', 'Vue d'ensemble du pr...', 'Paramètres', 'Raccourcis de projet', 'Authentication', 'Firestore', 'Catégories de produits', 'Bases de données et st...', 'Sécurité', 'Services d'IA', 'Hébergement et sans s...', and 'DevOps et engagement'. The main area is titled 'Authentication' and has tabs for 'Utilisateurs', 'Méthode de connexion', 'Modèles', 'Utilisation', and 'Paramètres'. A warning message states: 'Les fonctionnalités d'authentification suivantes ne seront plus disponibles lorsque Firebase Dynamic Links sera abandonné : l'authentification via un lien envoyé par e-mail pour les applications mobiles, ainsi que la compatibilité de Cordova OAuth avec les applications Web.' Below this is a search bar 'Recherchez par adresse e-mail, numéro de téléphone ou ID utilisateur' and a button 'Ajouter un utilisateur'. A table lists users with columns: 'Identifiant', 'Fournisseurs', 'Date de création', 'Dernière connexion', and 'ID utilisateur'. One user is listed: 'test@test.fr' with a mail icon, creation date '25 sept. 2026', and ID 'tGtKPX7C6bZVK8BpVZN9n9n...'. At the bottom, it shows 'Lignes par page : 50' and '1 - 1 of 1'.

Identifiant	Fournisseurs	Date de création	Dernière connexion	ID utilisateur
test@test.fr		25 sept. 2026		tGtKPX7C6bZVK8BpVZN9n9n...

1. Redirection et connexion

La redirection se fait dans le **main()** avec **authStateChanges()** qui lance **LoginTabBar** ou **HomePage** selon si l'utilisateur est connecté ou non :

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'home_page.dart';

FirebaseAuth auth = FirebaseAuth.instance;
FirebaseFirestore firestore = FirebaseFirestore.instance;

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  auth.authStateChanges().listen((User? user) {
    if (user == null) {
      print('Utilisateur non connecté');
      runApp(const LoginTabBar());
    } else {
      print('Utilisateur connecté: ' + user.email!);
      runApp(const HomePage());
    }
  });
}

class LoginTabBar extends StatelessWidget {
  const LoginTabBar({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DefaultTabController(
        length: 2,
```

La fonction **loginToFirebase()** utilise **signInWithEmailAndPassword()** avec le contenu des deux champs :

```
class LoginSection extends StatelessWidget {
  void loginToFirebase() {
    print(emailField.text.trim());
    print(passwordField.text.trim());
    try {
      auth
        .signInWithEmailAndPassword(
          email: emailField.text.trim(),
          password: passwordField.text.trim())
        .then((value) {
          print(value.toString());
        });
    } catch (e) {
      print(e.toString());
    }
  }
}

class SignUpSection extends StatelessWidget {
  SignUpSection({Key? key}) : super(key: key);
  final emailField = TextEditingController();
  final passwordField = TextEditingController();
  final pseudoField = TextEditingController();
  final birthdateField = TextEditingController();
  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(30),
      children: [
        TextField(
          // ...
        ),
      ],
    );
  }
}
```

On se connecte avec l'utilisateur de test :

9:41

Page de connexion

Connexion Inscription



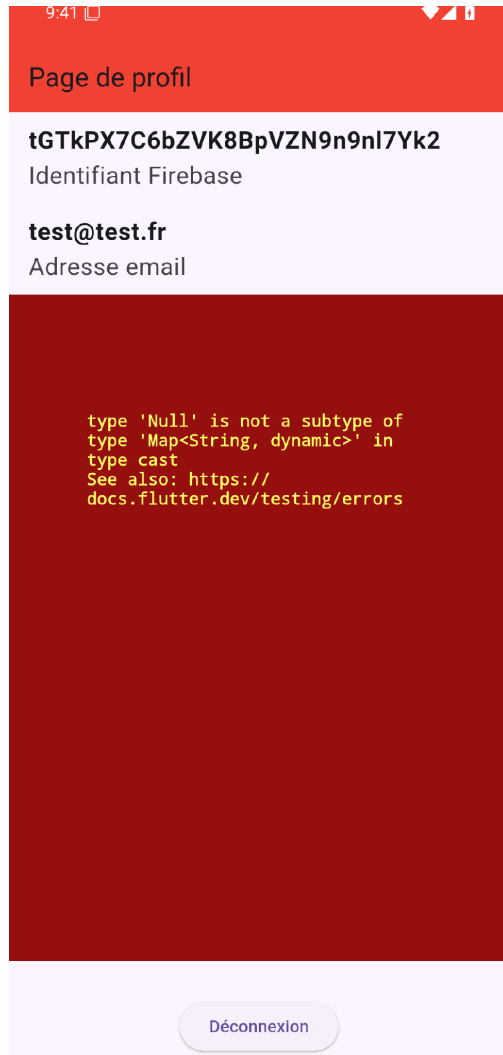
Email
test@test.fr

Mot de passe
.....

Connexion

1 2 3 4 5 6 7 8 9 0
q w e r t y u i o p
a s d f g h j k l
↑ z x c v b n m ↵
?123 , . ✓

On est bien redirigé vers la page de profil, mais le pseudo et la date de naissance font une erreur car cet utilisateur a été créé depuis la console, il n'a donc pas de document dans la collection **Users** :



Le bouton **Déconnexion** appelle `auth.signOut()` et nous renvoie sur la page de connexion.

2. Inscription et profil

Pour l'inscription on utilise `createUserWithEmailAndPassword()` puis `addUser()` qui crée un document dans la collection **Users** avec l'**uid** de l'utilisateur comme identifiant :

```
void signUpToFirebase() {
    print(emailField.text.trim());
    print(passwordField.text.trim());
    try {
        auth
            .createUserWithEmailAndPassword(
                email: emailField.text.trim(),
                password: passwordField.text.trim(),
            )
            .then((value) {
                print(value.user!.uid);
                addUser(
                    value.user!.uid,
                    pseudoField.text.trim(),
                    birthdateField.text.trim(),
                );
            });
    } catch (e) {
        print(e.toString());
    }
}

Future<void> addUser(String userID, String pseudo, String birthdate) {
    return firestore
        .collection('Users')
        .doc(userID)
        .set({
            'pseudo': pseudo,
            'birthdate': birthdate,
        })
        .then((value) => print("Utilisateur ajouté"))
        .catchError(
            (error) => print("Erreur: $error"),
        );
}
```

La page de profil affiche l'**uid** et l'email (Firebase Auth) puis le pseudo et la date de naissance avec le widget **GetUserData** qui lit le document Firestore :

```
}

class GetUserData extends StatelessWidget {
  final String documentId;
  final String fieldName;
  final String fieldTitle;
  const GetUserData(this.documentId, this.fieldName, this.fieldTitle);
  @override
  Widget build(BuildContext context) {
    CollectionReference users = firestore.collection('Users');
    return FutureBuilder<DocumentSnapshot>(
      future: users.doc(documentId).get(),
      builder:
        (BuildContext context, AsyncSnapshot<DocumentSnapshot> snapshot) {
          if (snapshot.hasError) {
            return const ListTile(
              title: Text(
                'Un problème est survenu',
                style: TextStyle(fontWeight: FontWeight.bold, fontSize: 20),
              ),
            );
          }
          if (snapshot.connectionState == ConnectionState.done) {
            Map<String, dynamic> data =
              snapshot.data!.data() as Map<String, dynamic>;
            return ListTile(
              title: Text(
                data[fieldName],
                style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 20),
              ),
              subtitle: Text(
                fieldTitle,
                style: const TextStyle(fontSize: 20),
              ),
            );
          }
          return const ListTile(
            title: Text(
              'En cours de chargement',
              style: TextStyle(fontWeight: FontWeight.bold, fontSize: 20),
            ),
          );
        },
    );
  },
}
```

On remplit le formulaire d'inscription :

Page de connexion

Connexion Inscription

Pseudo
Romain

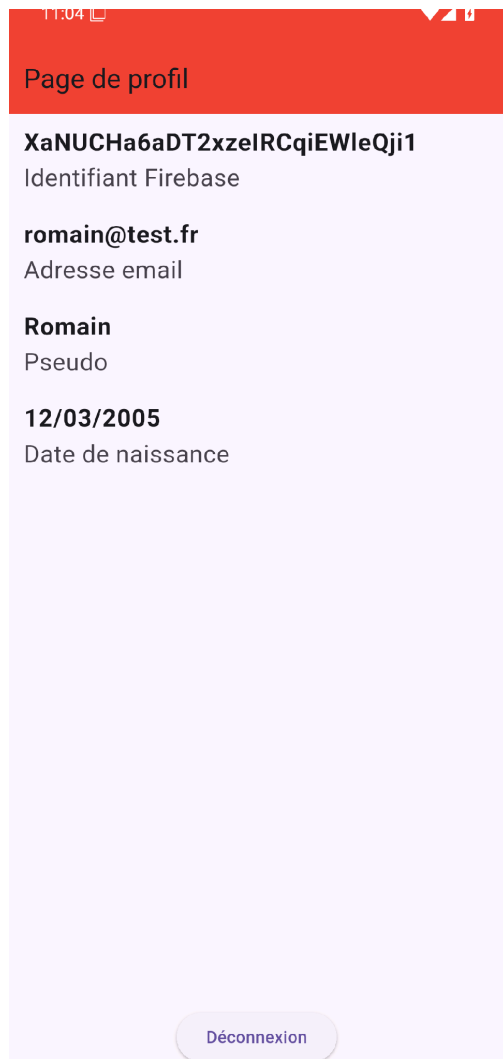
Date de naissance
12/03/2005

Email
romain@test.fr

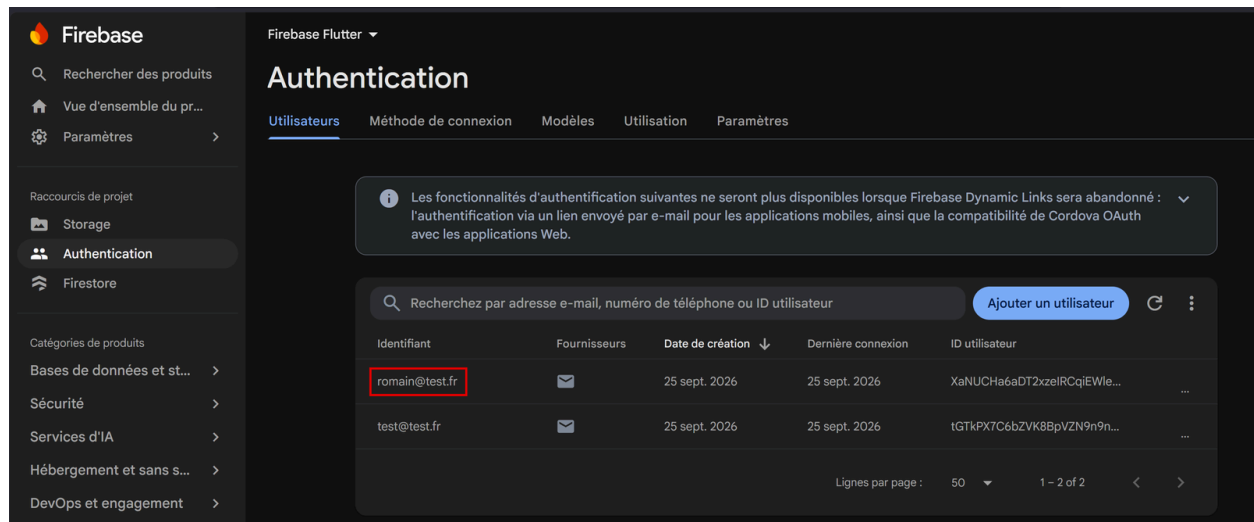
Mot de passe
.....

Inscription

Cette fois toutes les informations s'affichent :



On vérifie dans la console, le nouveau compte **romain@test.fr** apparaît dans **Authentication** :



Et un document avec son **uid** comme identifiant a été créé dans la collection **Users**, avec le pseudo et la date de naissance :

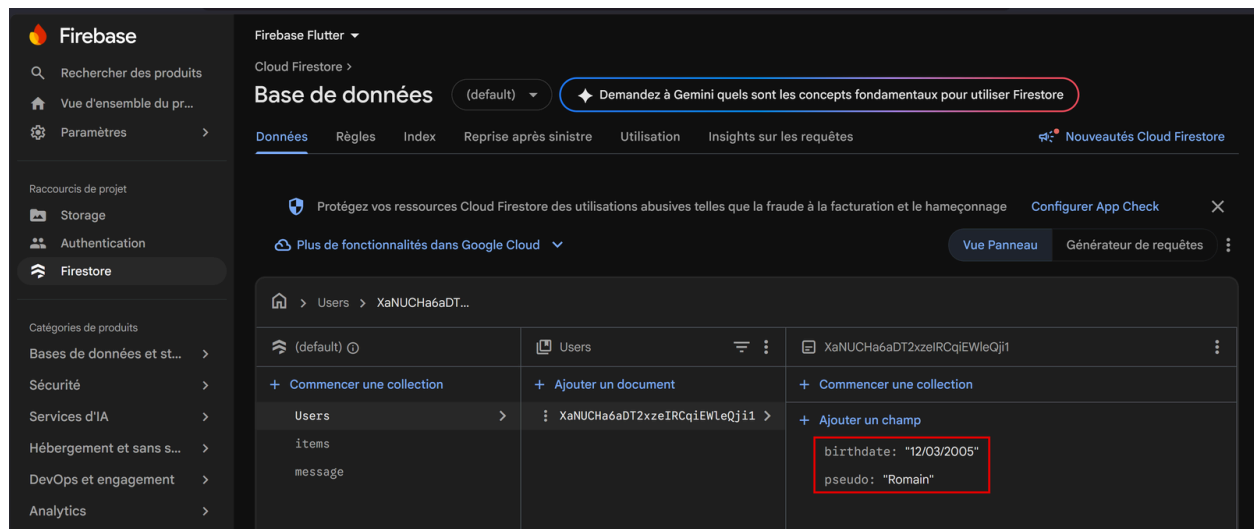


Photo de profil (Firebase Storage)

La dernière partie du TP utilise **Firebase Storage** pour stocker la photo de profil de l'utilisateur. Mais quand on veut l'activer, Firebase demande de passer au forfait **Blaze** qui nécessite un compte de facturation (carte bancaire) :

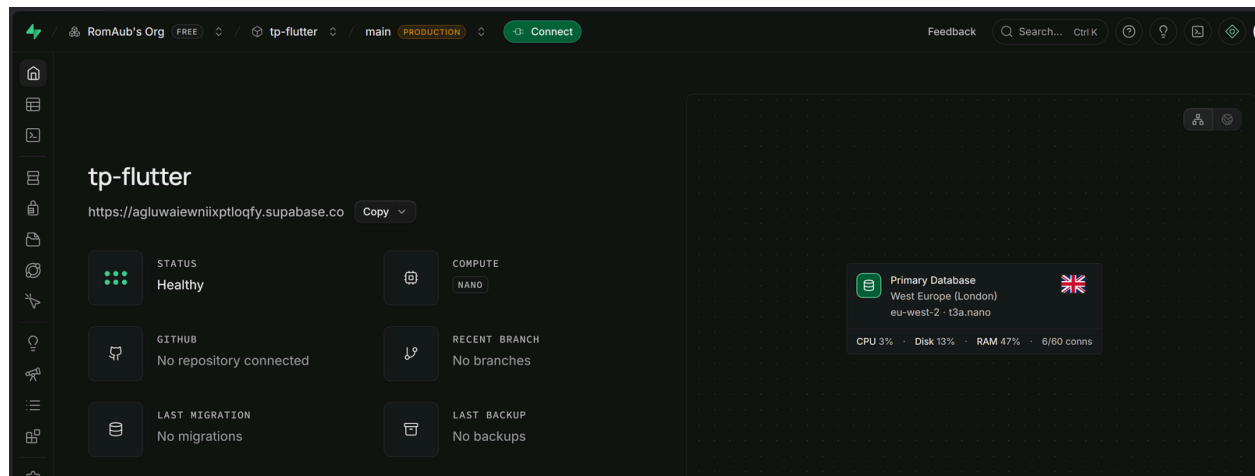


Storage n'est plus disponible avec le forfait gratuit **Spark**, on ne peut donc pas faire cette partie avec Firebase. Comme demandé on la refait avec **Supabase**.

Passage à Supabase

Supabase est une alternative open source à Firebase, la grosse différence c'est qu'il utilise une base **PostgreSQL** (donc du **SQL**) au lieu d'une base NoSQL.

On crée un compte sur *supabase.com* puis un projet “**tp-flutter**” en région Europe :



On récupère l’**URL** du projet et la clé **publishable** (anon) dans *Project Settings* → *API Keys*, qu’on met dans un fichier **config.dart**. Cette clé peut être dans l’application, contrairement à la clé **secret**.

Création des tables

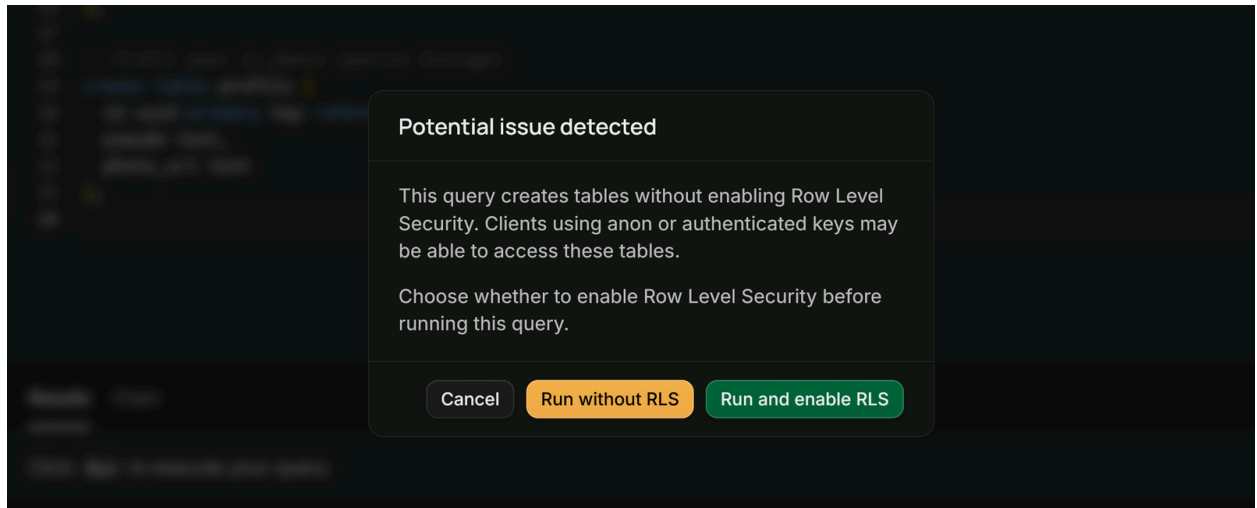
Dans le **SQL Editor** on crée les tables **groupes**, **contacts** (avec une clé étrangère vers groupes) et **profils** :

```
-- Niveau 1 et 2 : tables contacts et groupes (cle etrangere)
create table groupes (
  id bigint generated always as identity primary key,
  nom text not null,
  proprietaire uuid references auth.users(id) default auth.uid()
);

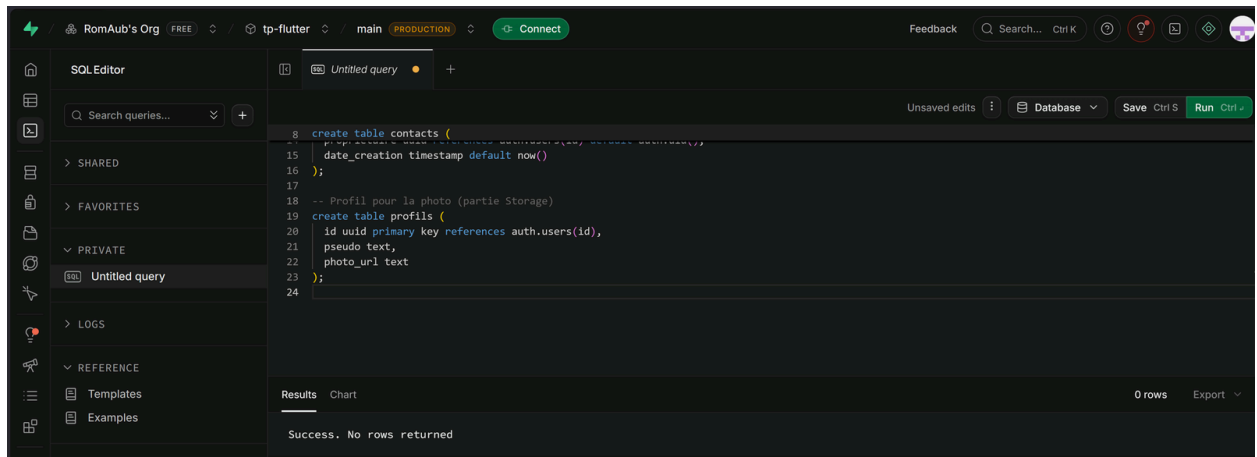
create table contacts (
  id uuid primary key default gen_random_uuid(),
  nom text not null,
  telephone text,
  age int,
  groupe_id bigint references groupes(id) on delete set null,
  proprietaire uuid references auth.users(id) default auth.uid(),
  date_creation timestamp default now()
);

-- Profil pour la photo (partie Storage)
create table profils (
  id uuid primary key references auth.users(id),
  pseudo text,
  photo_url text
);
```

Supabase nous prévient que les tables n'ont pas de **RLS** (Row Level Security), on l'activera juste après :



La requête s'exécute sans erreur :

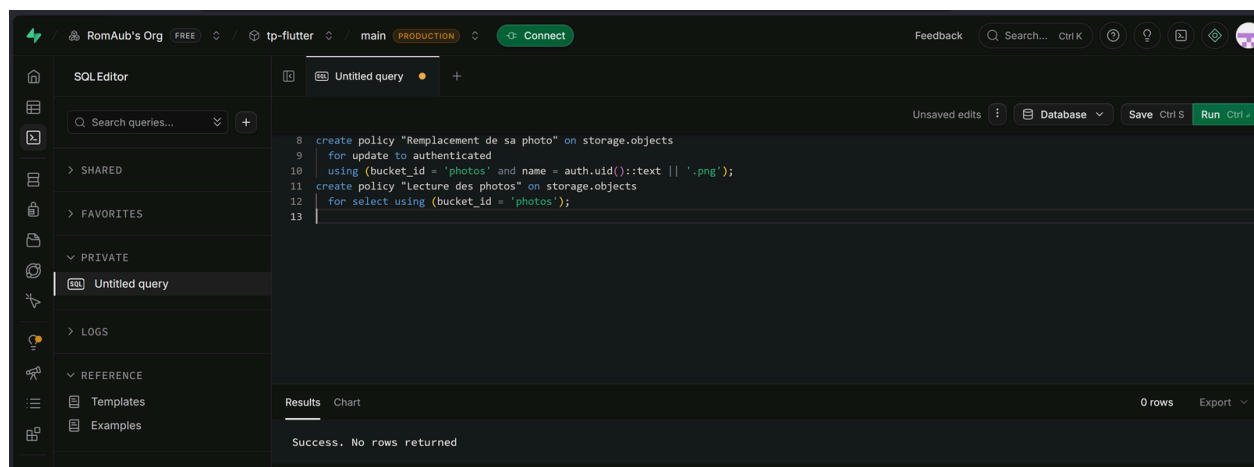


Storage

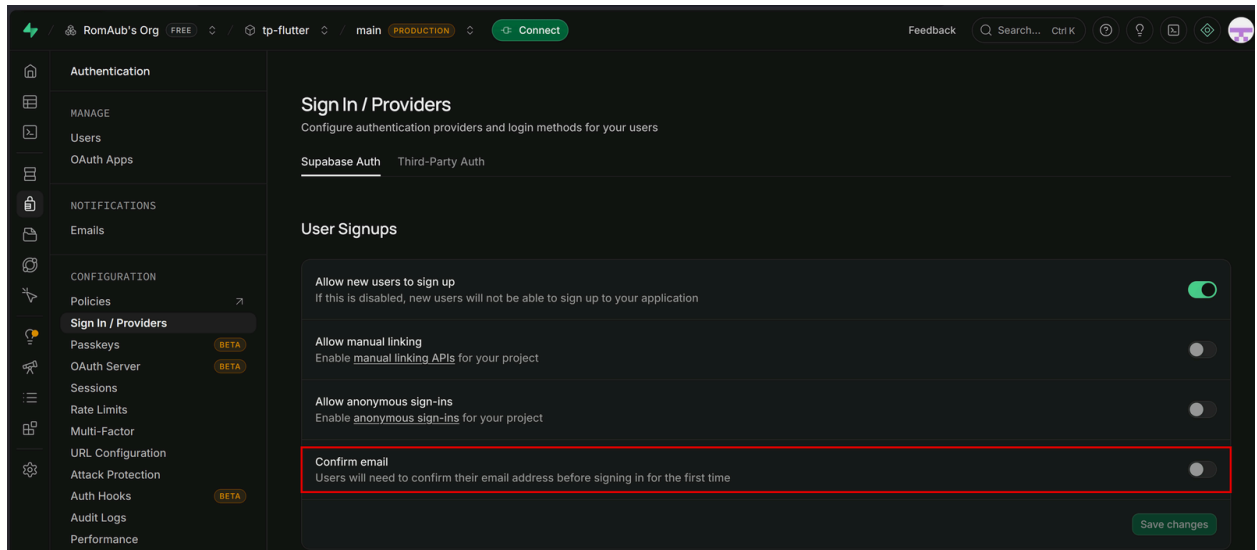
On crée le bucket “**photos**” et ses politiques d’accès : un utilisateur connecté peut envoyer seulement sa propre photo (nommée avec son **id**) et tout le monde peut les lire :

```
❏ Connect to PostgreSQL
|-- Bucket "photos" pour les photos de profil (public en lecture)
insert into storage.buckets (id, name, public) values ('photos', 'photos', true);

-- Un utilisateur connecte peut envoyer / remplacer uniquement SA photo (nom = son id)
create policy "Envoi de sa photo" on storage.objects
  for insert to authenticated
  with check (bucket_id = 'photos' and name = auth.uid()::text || '.png');
create policy "Remplacement de sa photo" on storage.objects
  for update to authenticated
  using (bucket_id = 'photos' and name = auth.uid()::text || '.png');
create policy "Lecture des photos" on storage.objects
  for select using (bucket_id = 'photos');
```



Il a aussi fallu désactiver “**Confirm email**” dans **Authentication** pour pouvoir créer des comptes de test sans vraie adresse mail :



Application photo de profil

Dans le nouveau projet on ajoute **supabase_flutter** et **image_picker** :

```
# the latest version available on pub.dev. To see which dependencies have newer
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  supabase_flutter: ^2.17.2
  image_picker: ^1.2.3

dev_dependencies:
  flutter_test:
    sdk: flutter

  # The "flutter_lints" package below contains a set of recommended lints to
  # encourage good coding practices. The lint set provided by the package is
  # activated in the `analysis_options.yaml` file located at the root of your
  # package. See that file for information about deactivating specific lint
  # rules and activating additional ones.
  flutter_lints: ^6.0.0

# For information on the generic Dart part of this file, see the
# following page: https://dart.dev/tools/pub/pubspec

# The following section is specific to Flutter packages.
flutter:

  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons without
```

Supabase s'initialise dans le **main()** avec l'URL et la clé, la redirection se fait avec **onAuthStateChange** :

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'config.dart';

final supabase = Supabase.instance.client;

Run | Debug | Profile
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Supabase.initialize(
    url: supabaseUrl,
    anonKey: supabaseAnonKey,
  );
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: StreamBuilder<AuthState>(
        stream: supabase.auth.onAuthStateChange,
        builder: (context, snapshot) {
          if (supabase.auth.currentSession == null) {
            return const LoginPage();
          }
          return const ProfilPage();
        }
      )
    );
  }
}
```

La fonction **uploadFile()** envoie l'image dans le bucket avec **upload()** (**upsert** pour remplacer l'ancienne) puis enregistre l'URL publique dans la table **profils**. C'est l'équivalent de **putFile()** et **getDownloadURL()** de Firebase :

```
// Equivalent de putFile() de Firebase Storage
Future uploadFile() async {
  await supabase.storage.from('photos').upload(
    '$userID.png',
    _image!,
    fileOptions: const FileOptions(upsert: true),
  );
  final url = supabase.storage.from('photos').getPublicUrl('$userID.png');
  // On stocke seulement l'url dans la table profils
  await supabase.from('profils').upsert({'id': userID, 'photo_url': url});
  print('Photo de profil mise à jour');
  refreshPage(context);
}
```

Le widget **ImagePicture** relit l'URL et affiche la photo dans un **CircleAvatar** :

```
}

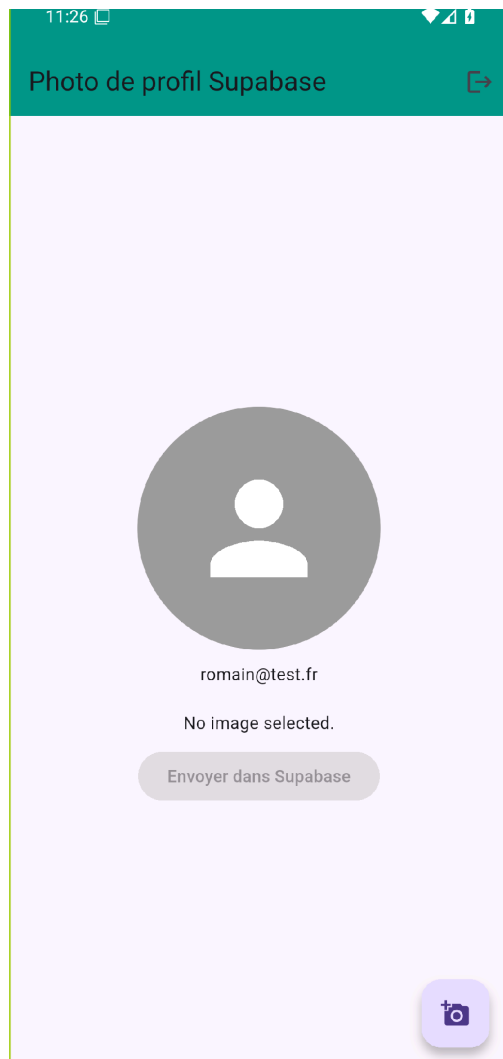
class _ImagePictureState extends State<ImagePicture> {
  String? userPhotoUrl;

  @override
  void initState() {
    super.initState();
    getProfilImage();
  }

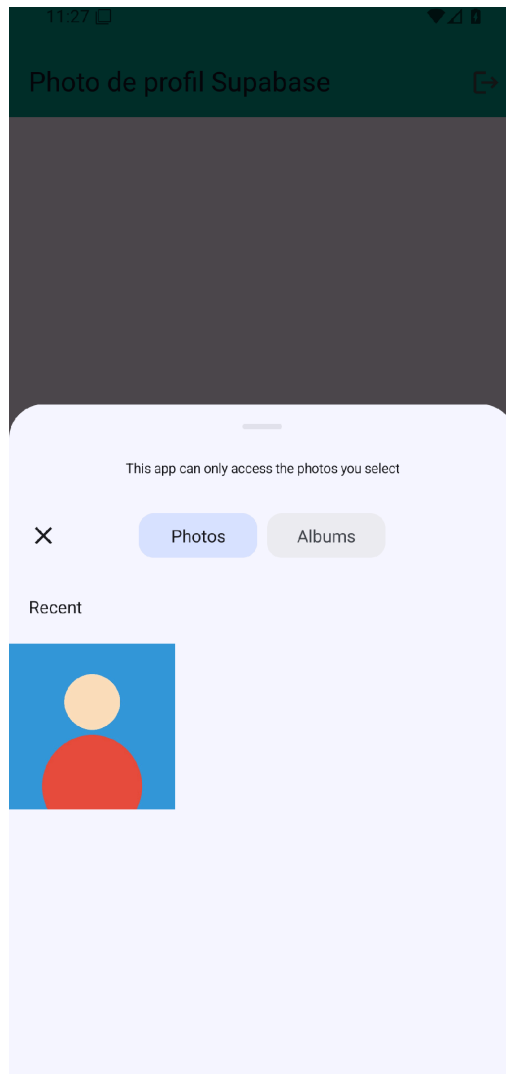
  // On relit l'url dans la table profils (equivalent de getDownloadURL())
  getProfilImage() async {
    final data = await supabase
      .from('profils')
      .select('photo_url')
      .eq('id', widget.userID)
      .maybeSingle();
    if (data != null && data['photo_url'] != null) {
      setState(() {
        // le ?t= evite que l'ancienne photo reste en cache
        userPhotoUrl =
          '${data['photo_url']}?t=${DateTime.now().millisecondsSinceEpoch}';
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 200,
      width: 200,
      child: CircleAvatar(
        backgroundColor: Colors.grey,
        backgroundImage:
          userPhotoUrl == null ? null : NetworkImage(userPhotoUrl!),
        child: userPhotoUrl == null
          ? const Icon(Icons.person, size: 120, color: Colors.white)
          : null,
      ),
    );
  }
}
```

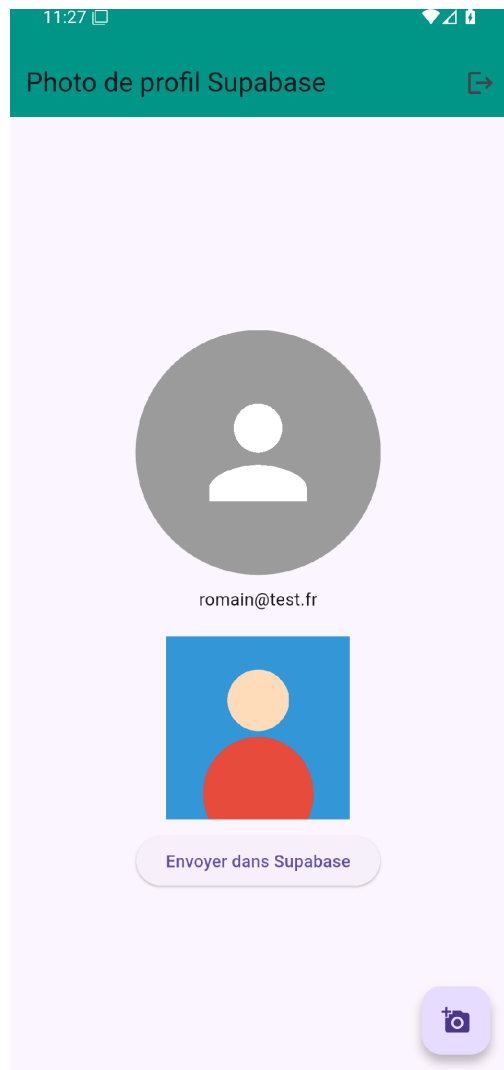
Une fois connecté, on a l'avatar par défaut :



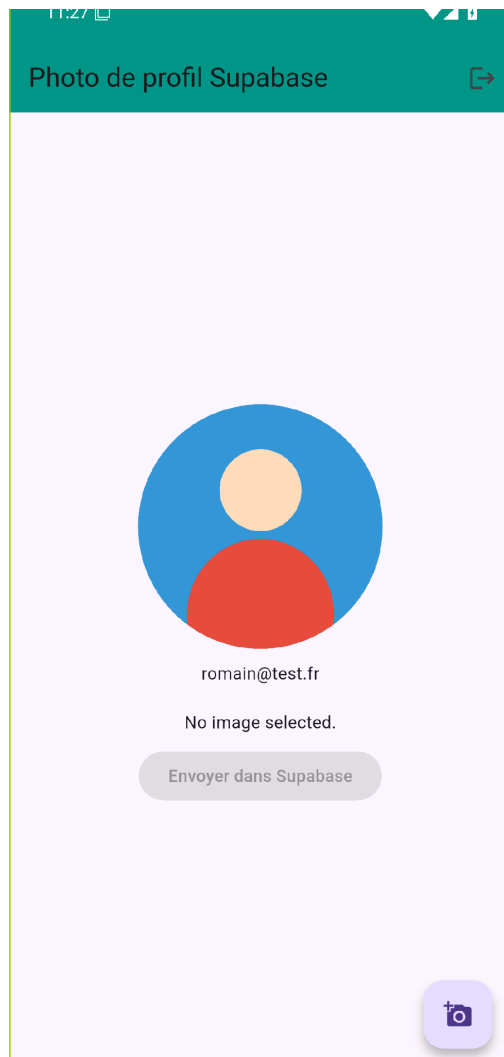
On choisie une image dans la galerie avec le bouton en bas à droite :



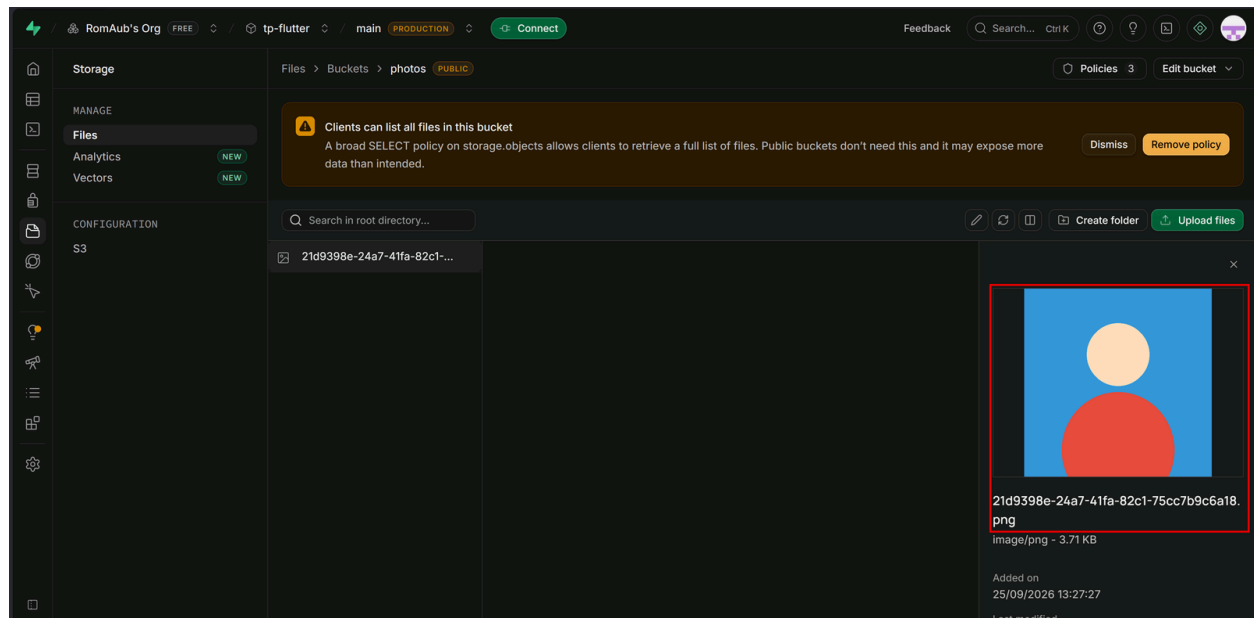
L'image sélectionnée s'affiche, on clique sur “**Envoyer dans Supabase**” :



La photo de profil est maintenant celle stockée dans Supabase :



Dans **Storage**, la photo est bien dans le bucket **photos** avec l'**uid** de l'utilisateur comme nom de fichier :



Supabase nous prévient que la politique de lecture permet de lister tous les fichiers du bucket. Comme le bucket est public ce n'est pas grave pour le TP, mais sur un vrai projet il faudrait la restreindre.

Compléter le profil

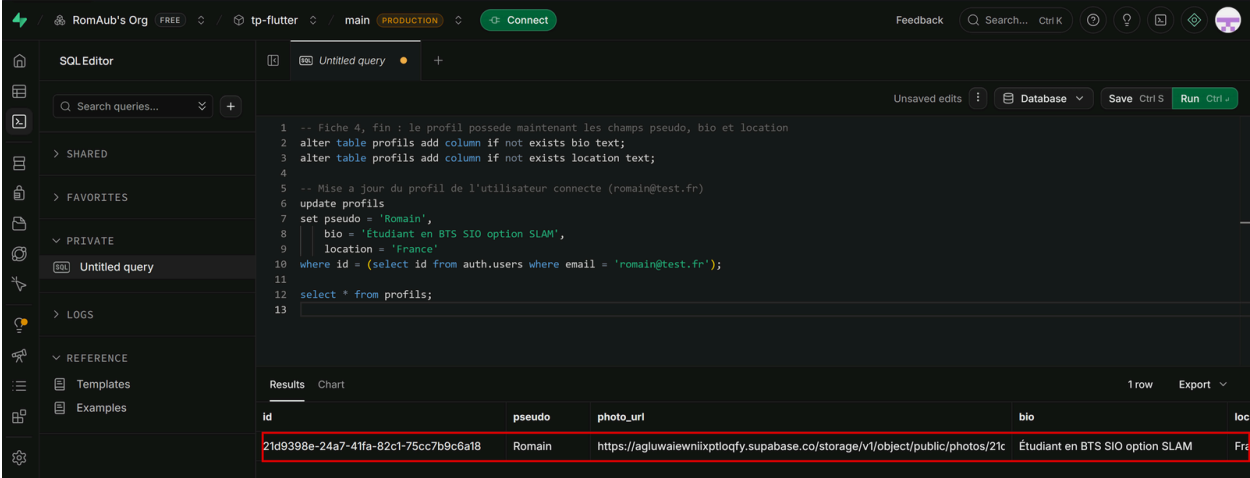
Pour finir la fiche 4, on complète le profil comme dans le tuto : le profil possède maintenant les champs **pseudo**, **bio** et **location**. On ajoute les deux colonnes qui manquent et on met à jour le profil de l'utilisateur connecté dans le **SQL Editor** :

```
-- Fiche 4, fin : le profil possède maintenant les champs pseudo, bio et location
alter table profils add column if not exists bio text;
alter table profils add column if not exists location text;

-- Mise a jour du profil de l'utilisateur connecte (romain@test.fr)
update profils
set pseudo = 'Romain',
    bio = 'Étudiant en BTS SIO option SLAM',
    location = 'France'
where id = (select id from auth.users where email = 'romain@test.fr');

select * from profils;
```

Le profil de **romain@test.fr** a bien ses trois champs remplis :



The screenshot shows a SQL Editor interface with a sidebar on the left containing navigation options like 'SHARED', 'FAVORITES', 'PRIVATE', 'LOGS', and 'REFERENCE'. The main area displays a SQL query in a dark theme. The query includes comments and SQL commands to alter a table, update a profile, and select data. Below the query, the 'Results' tab is active, showing a single row of data for a user profile. The row is highlighted with a red border.

```
1 -- Fiche 4, fin : le profil possède maintenant les champs pseudo, bio et location
2 alter table profiles add column if not exists bio text;
3 alter table profiles add column if not exists location text;
4
5 -- Mise à jour du profil de l'utilisateur connecté (romain@test.fr)
6 update profiles
7 set pseudo = 'Romain',
8     bio = 'Étudiant en BTS SIO option SLAM',
9     location = 'France'
10 where id = (select id from auth.users where email = 'romain@test.fr');
11
12 select * from profiles;
13
```

id	pseudo	photo_url	bio	location
21d9398e-24a7-41fa-82c1-75cc7b9c6af8	Romain	https://agluwaiewnliixptloqfy.supabase.co/storage/v1/object/public/photos/21c	Étudiant en BTS SIO option SLAM	France

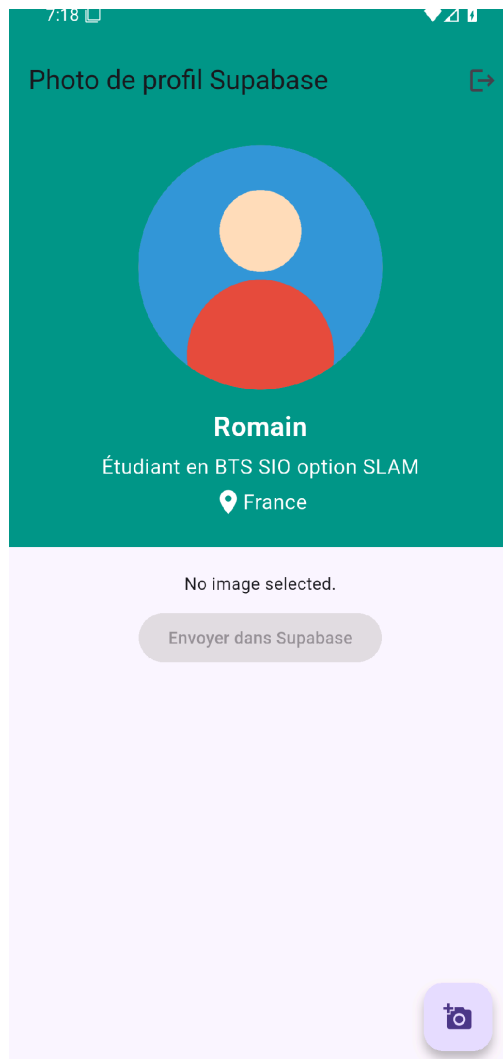
On reprend le widget **GetUserData** du tuto précédent, adapté à Supabase : il lit un champ de la table **profils** avec un **FutureBuilder** (équivalent du *users.doc(userID).get()* de Firestore) :

```
// Reprise du widget GetUserData : lit un champ du profil dans la table profils
class GetUserData extends StatelessWidget {
  final String userID;
  final String fieldName;
  final TextStyle fieldStyle;
  const GetUserData({
    Key? key,
    required this.userID,
    required this.fieldName,
    required this.fieldStyle,
  }) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return FutureBuilder<Map<String, dynamic>?>(
      future: supabase
        .from('profils')
        .select(fieldName)
        .eq('id', userID)
        .maybeSingle(),
      builder: (BuildContext context,
        AsyncSnapshot<Map<String, dynamic>?> snapshot) {
        if (snapshot.hasError) {
          return Text(
            'Un problème est survenu',
            style: fieldStyle,
          );
        }
        if (snapshot.connectionState == ConnectionState.done) {
          Map<String, dynamic>? data = snapshot.data;
          return Text(
            data?[fieldName] ?? '',
            style: fieldStyle,
          );
        }
        return Text(
          'En cours de chargement',
          style: fieldStyle,
        );
      },
    );
  }
}
```

La section **ProfilTextSection** affiche le pseudo, la bio et la localisation sous la photo :

```
// Section texte du profil : pseudo, bio et localisation
class ProfilTextSection extends StatelessWidget {
  final String userID;
  const ProfilTextSection({Key? key, required this.userID}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        GetUserData(
          userID: userID,
          fieldName: 'pseudo',
          fieldStyle: const TextStyle(
            color: Colors.white,
            fontSize: 22,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 5),
        GetUserData(
          userID: userID,
          fieldName: 'bio',
          fieldStyle: const TextStyle(
            color: Colors.white,
            fontSize: 17,
          ),
        ),
        const SizedBox(height: 5),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.location_on,
              color: Colors.white,
            ),
            GetUserData(
              userID: userID,
              fieldName: 'location',
              fieldStyle: const TextStyle(
                color: Colors.white,
                fontSize: 17,
              ),
            ),
          ],
        ),
      ],
    ),
  ],
);
```

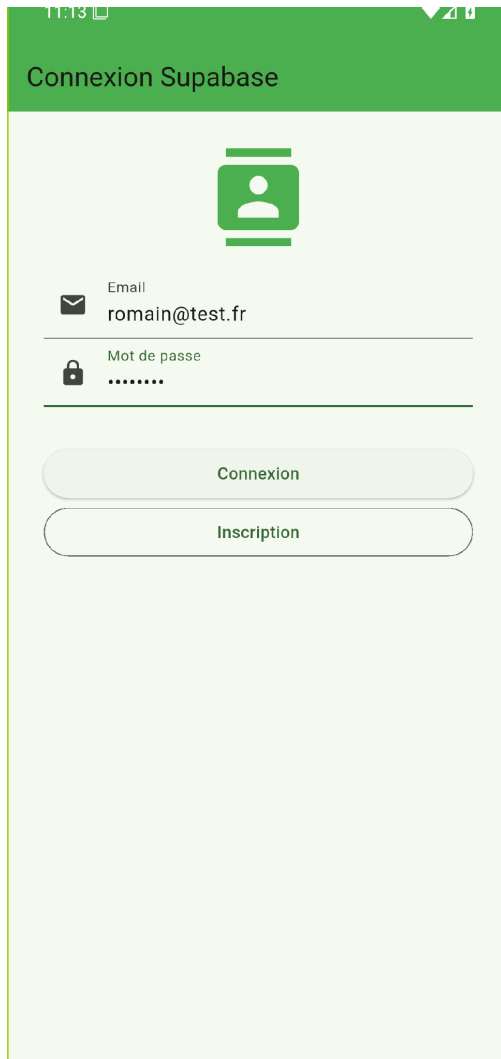
Voilà le rendu final du profil, avec la photo stockée dans Supabase Storage et les informations de la table **profils** :



Exercices Supabase

Niveau 1 : Carnet de contacts (CRUD)

Application carnet de contacts avec authentification email / mot de passe et une table PostgreSQL. On crée un compte avec **signUp()** :



Connexion Supabase

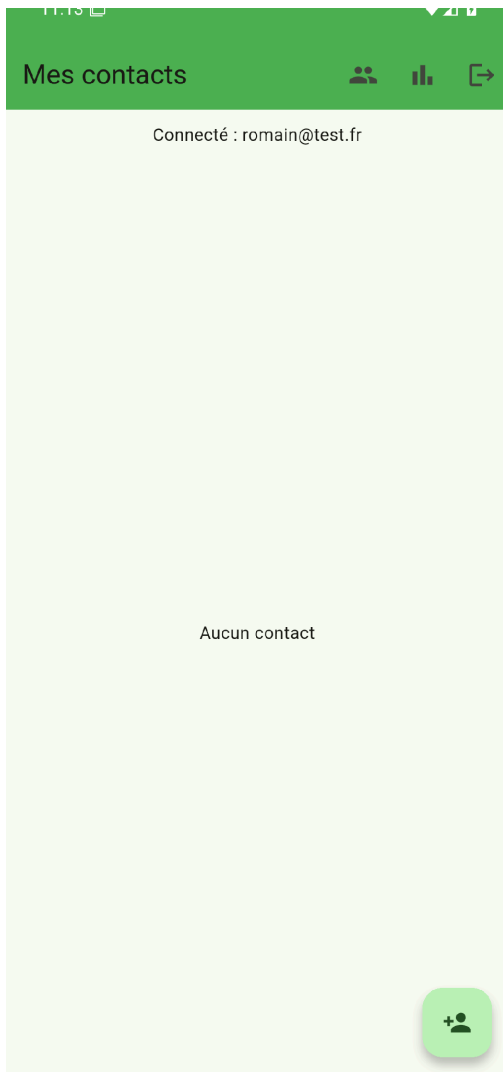


Email
romain@test.fr

Mot de passe
.....

Connexion

Inscription



Les fonctions **CRUD** : **insert()**, **update()** et **delete()** (la lecture se fait avec le stream du niveau 4) :

```
}  
  
// CREATE  
Future<void> ajouterContact(  
    String nom, String telephone, int age, int? groupId) async {  
    await supabase.from('contacts').insert({  
        'nom': nom,  
        'telephone': telephone,  
        'age': age,  
        'groupe_id': groupId,  
        'proprietaire': supabase.auth.currentUser!.id,  
    });  
}  
  
// UPDATE  
Future<void> modifierAge(String id, int nouvelAge) async {  
    await supabase.from('contacts').update({'age': nouvelAge}).eq('id', id);  
}  
  
// DELETE  
Future<void> supprimerContact(String id) async {  
    await supabase.from('contacts').delete().eq('id', id);  
}  
  
// Niveau 5 : appel de l'Edge Function  
Future<void> afficherStatistique() async {  
    final reponse = await supabase.functions.invoke('contacts-majeurs');  
    print(reponse.data);  
    if (!mounted) return;  
    showDialog(  
        context: context,  
        builder: (context) => AlertDialog(  
            title: const Text('Statistique (Edge Function)'),  
            content: Text(  
                'Contacts majeurs : ${reponse.data['majeurs']} / ${reponse.data['total']}',  
            ),  
        ),  
    );  
}
```

Ajout d'un contact, on peut choisir son groupe :

Mes contacts

Connecté : romain@test.fr

Nouveau contact

Nom
Lucas Martin

Téléphone
0612345678

Âge
19

BTS SIO

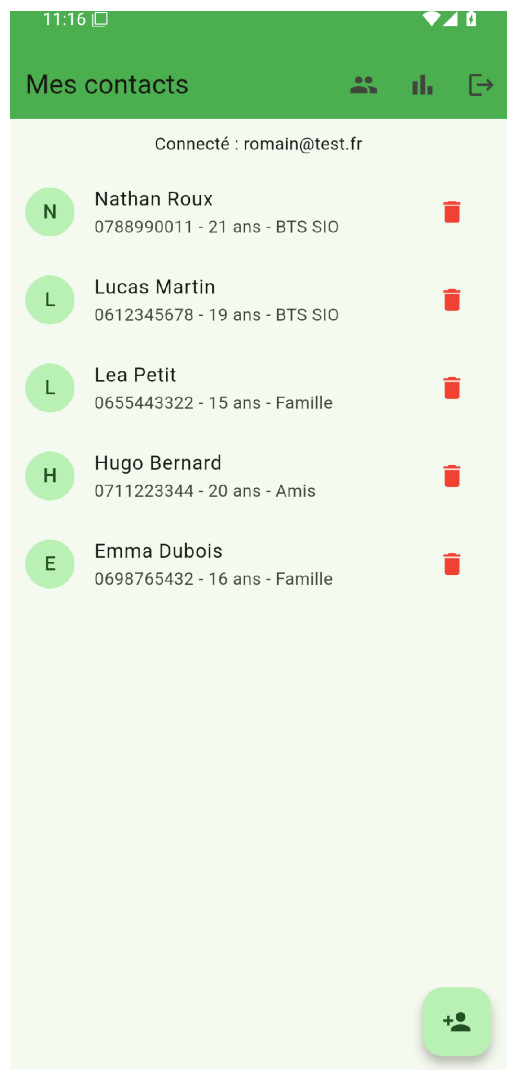
Valider

1 2 3 -

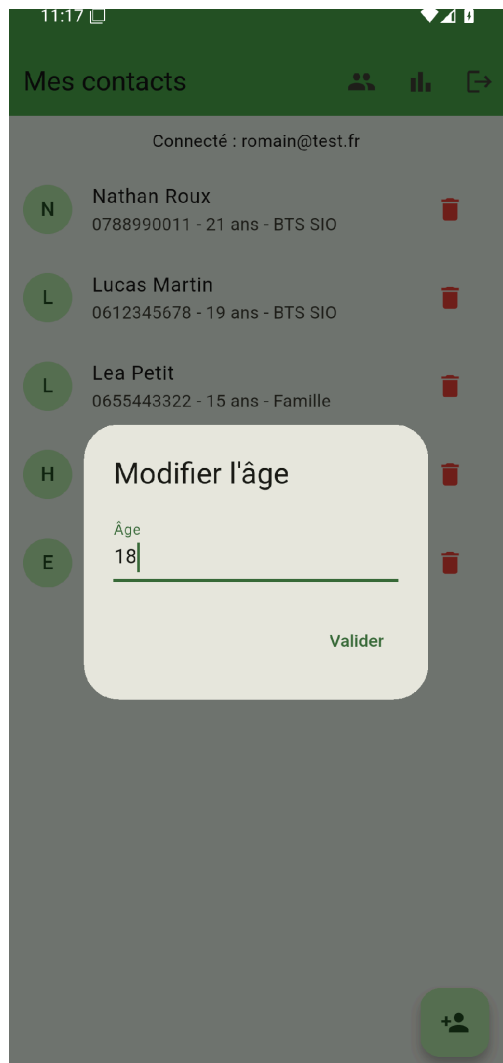
4 5 6 _

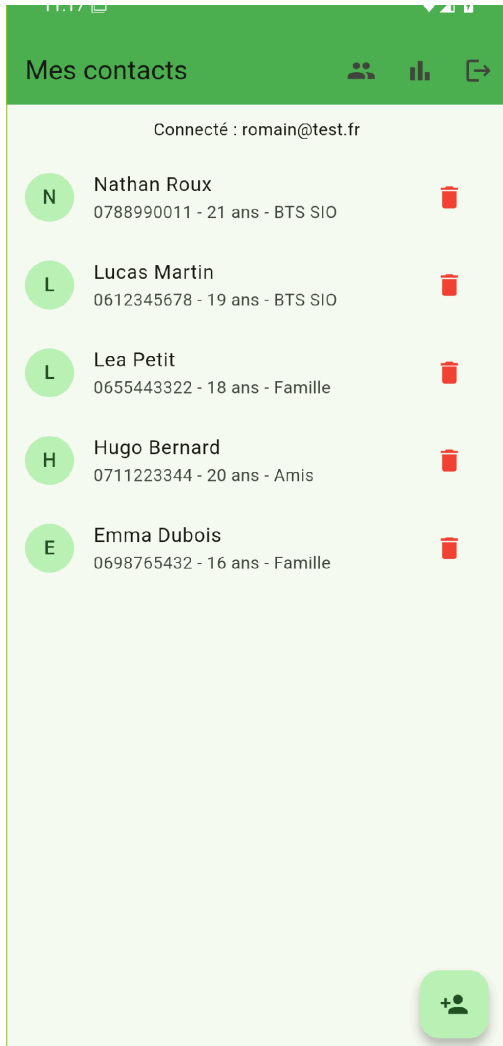
7 8 9 ✕

, 0 . ✓

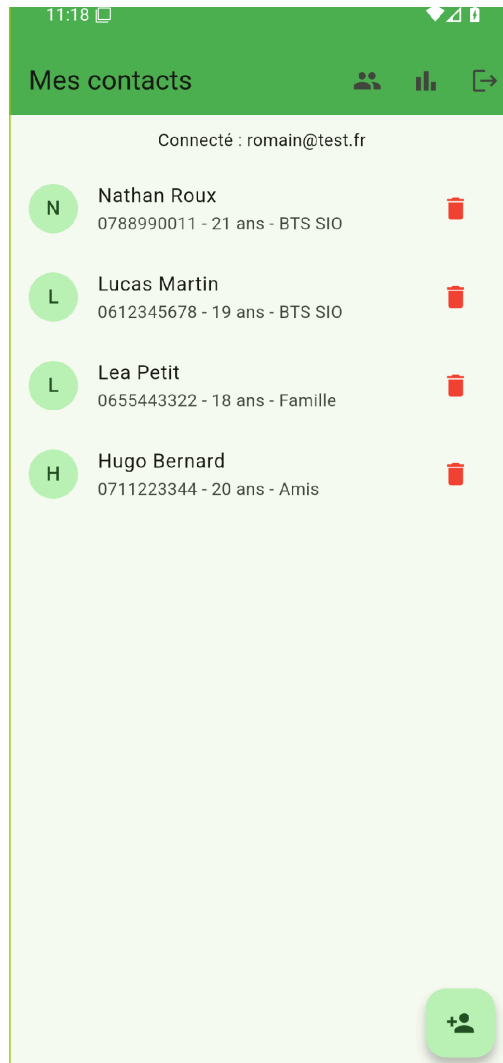


Modification de l'âge de **Lea Petit** (15 → 18 ans) :





Suppression d'un contact avec la corbeille (on l'a fait à la fin, après les tests des niveaux 2 et 5, c'est pour ça qu'**Emma Dubois** apparait encore dans ces captures) :



Niveau 2 : Relationnel

La table **contacts** a une clé étrangère **groupe_id** vers **groupes**. Avec Supabase la jointure se fait directement dans le **select()** : `select('id, nom, contacts(nom, age)')` :

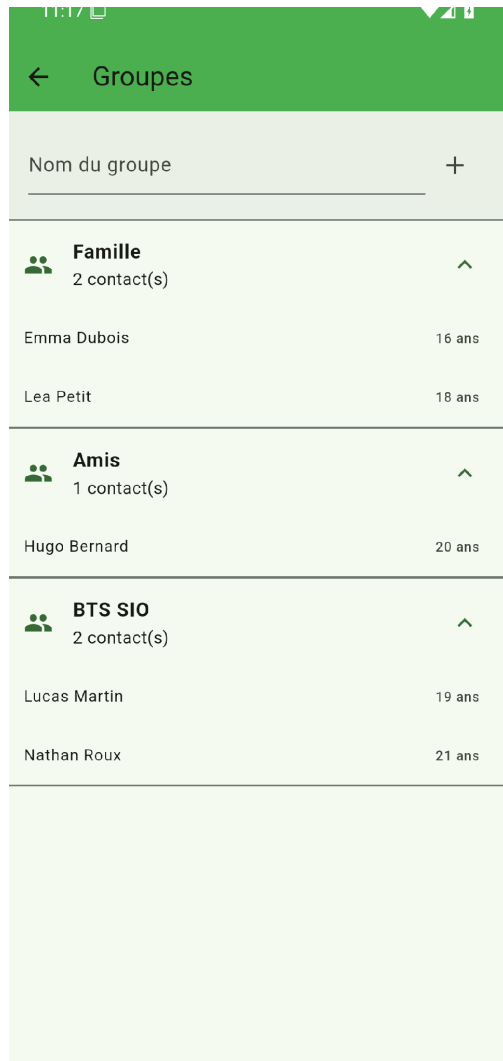
```
class _GroupesPageState extends State<GroupesPage> {
  void initState() {
  }

  // Jointure : chaque groupe avec ses contacts
  Future<void> chargerGroupes() async {
    final data =
      await supabase.from('groupes').select('id, nom, contacts(nom, age)');
    setState(() => groupes = data);
  }

  Future<void> ajouterGroupe() async {
    await supabase.from('groupes').insert({
      'nom': nomField.text.trim(),
      'proprietaire': supabase.auth.currentUser!.id,
    });
    nomField.clear();
    chargerGroupes();
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Groupes'),
      backgroundColor: Colors.green,
    ), // AppBar
    body: Column(
      children: [
        Container(
          height: 100,
          width: 100,
          color: Colors.grey,
        ),
      ],
    ),
  );
}
```

La page **Groupes** affiche chaque groupe avec ses contacts :



Niveau 3 : Sécurité (RLS)

On active la **Row Level Security** et on crée les politiques : chaque utilisateur peut lire, ajouter, modifier ou supprimer seulement ses propres lignes (`auth.uid() = propriétaire`) :

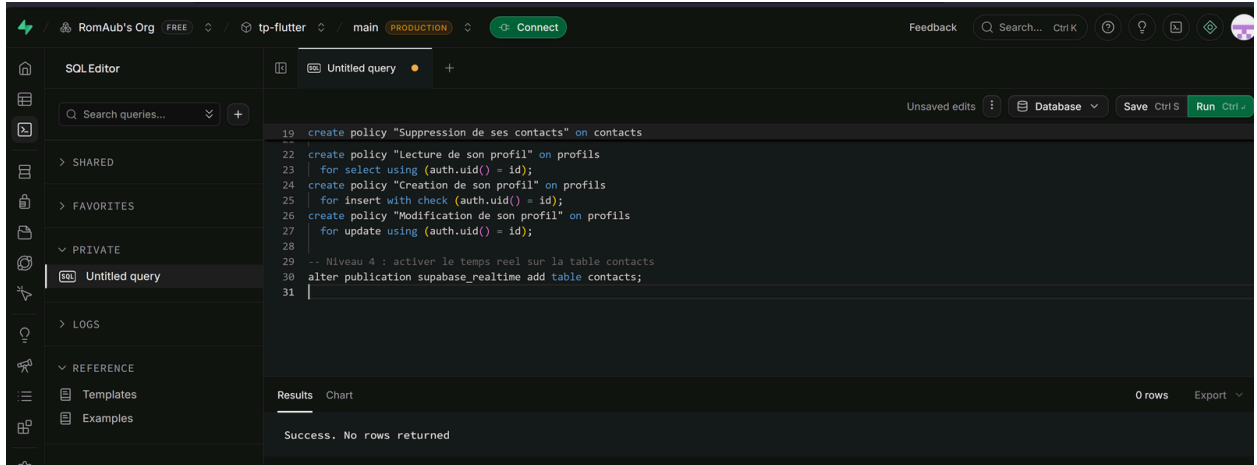
```
-- Niveau 3 : Row Level Security, chaque utilisateur ne voit que ses donnees
alter table groupes enable row level security;
alter table contacts enable row level security;
alter table profils enable row level security;

create policy "Lecture de ses propres groupes" on groupes
  for select using (auth.uid() = propriétaire);
create policy "Insertion groupe si connecte" on groupes
  for insert with check (auth.uid() = propriétaire);
create policy "Suppression de ses groupes" on groupes
  for delete using (auth.uid() = propriétaire);

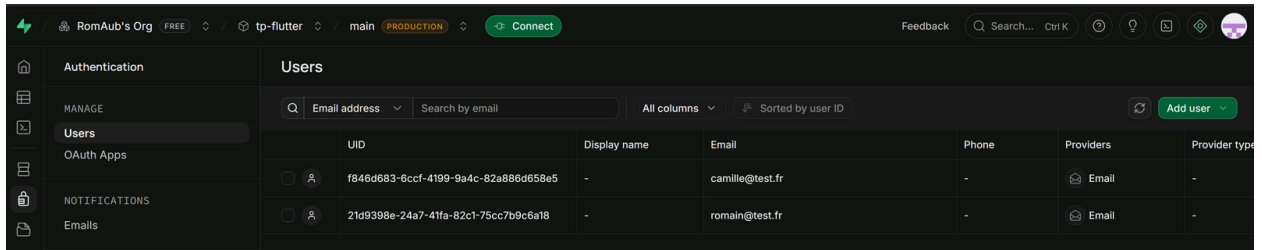
create policy "Lecture de ses propres contacts" on contacts
  for select using (auth.uid() = propriétaire);
create policy "Insertion contact si connecte" on contacts
  for insert with check (auth.uid() = propriétaire);
create policy "Modification de ses contacts" on contacts
  for update using (auth.uid() = propriétaire);
create policy "Suppression de ses contacts" on contacts
  for delete using (auth.uid() = propriétaire);

create policy "Lecture de son profil" on profils
  for select using (auth.uid() = id);
create policy "Creation de son profil" on profils
  for insert with check (auth.uid() = id);
create policy "Modification de son profil" on profils
  for update using (auth.uid() = id);

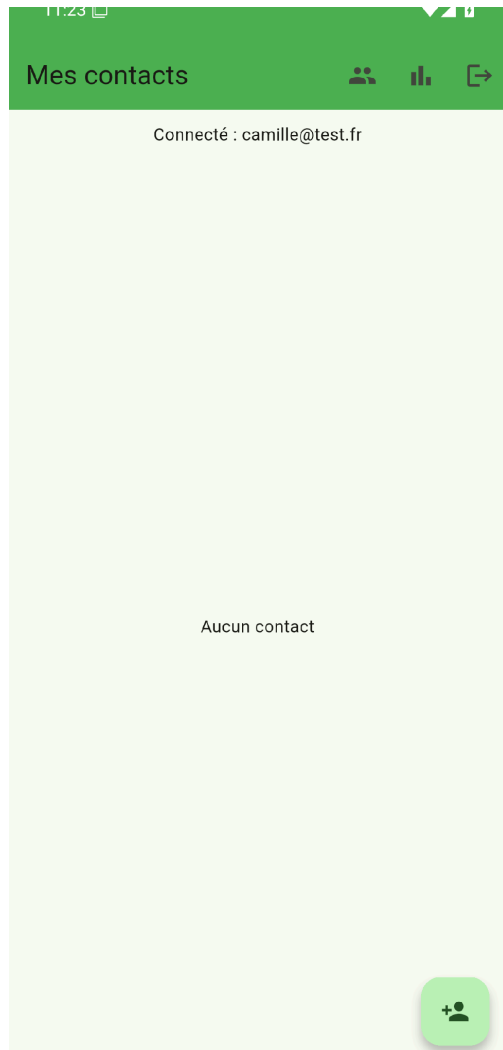
-- Niveau 4 : activer le temps reel sur la table contacts
alter publication supabase_realtime add table contacts;
```



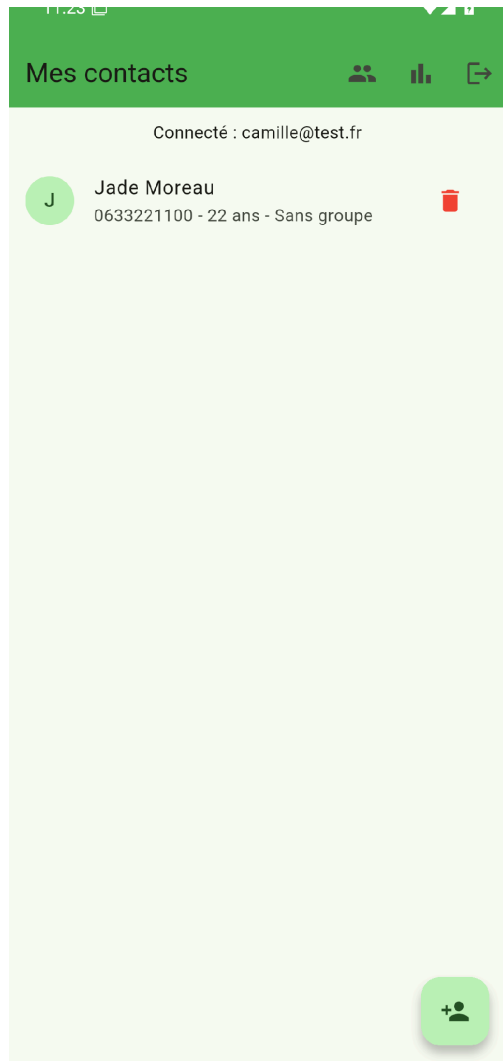
On a maintenant deux comptes dans **Authentication** :



Test avec un deuxième compte **camille@test.fr** : elle ne voit aucun des contacts de **romain@test.fr** :



Et elle voit seulement ceux qu'elle ajoute :



Dans le **Table Editor** (en admin, donc sans la RLS) on voit bien tous les contacts, et celui de Camille a un **proprietaire** différent des autres :

id	nom	telephone	age	groupe_id	proprietaire
1cd8fd2e-14bd-4063-b2d3-7dc0081f9	Jade Moreau	0633221100	22	NULL	f846d683-6ccf-41
3c5db48c-0690-49c0-870e-7abed77t	Lucas Martin	0612345678	19	3	21d9398e-24a7-4
8c5dca71-4d75-436b-a26e-c688ec13t	Hugo Bernard	0711223344	20	2	21d9398e-24a7-4
ae956565-d433-4d81-aaec-ae3d5852	Nathan Roux	0788990011	21	3	21d9398e-24a7-4
f1cd86a8-c6c8-4ee2-985b-0bf298e3t	Lea Petit	0655443322	18	1	21d9398e-24a7-4

Niveau 4 : Temps réel

La liste utilise `.stream(primaryKey: ['id'])` dans un **StreamBuilder** (le temps réel est activé sur la table avec `alter publication supabase_realtime`, voir le script RLS). Quand on a modifié l'âge au niveau 1, la liste s'est mise à jour toute seule sans recharger :

```
const ContactsPage({key: Key}) : Super({key: Key}),
@override
State<ContactsPage> createState() => _ContactsPageState();
}

class _ContactsPageState extends State<ContactsPage> {
  // Niveau 4 : flux temps réel de la table contacts
  final contactsStream =
    supabase.from('contacts').stream(primaryKey: ['id']).order('nom');
  List<Map<String, dynamic>> groupes = [];

  @override
  void initState() {
    super.initState();
    chargerGroupes();
  }

  Future<void> chargerGroupes() async {
    final data = await supabase.from('groupes').select().order('nom');
    setState(() => groupes = data);
  }

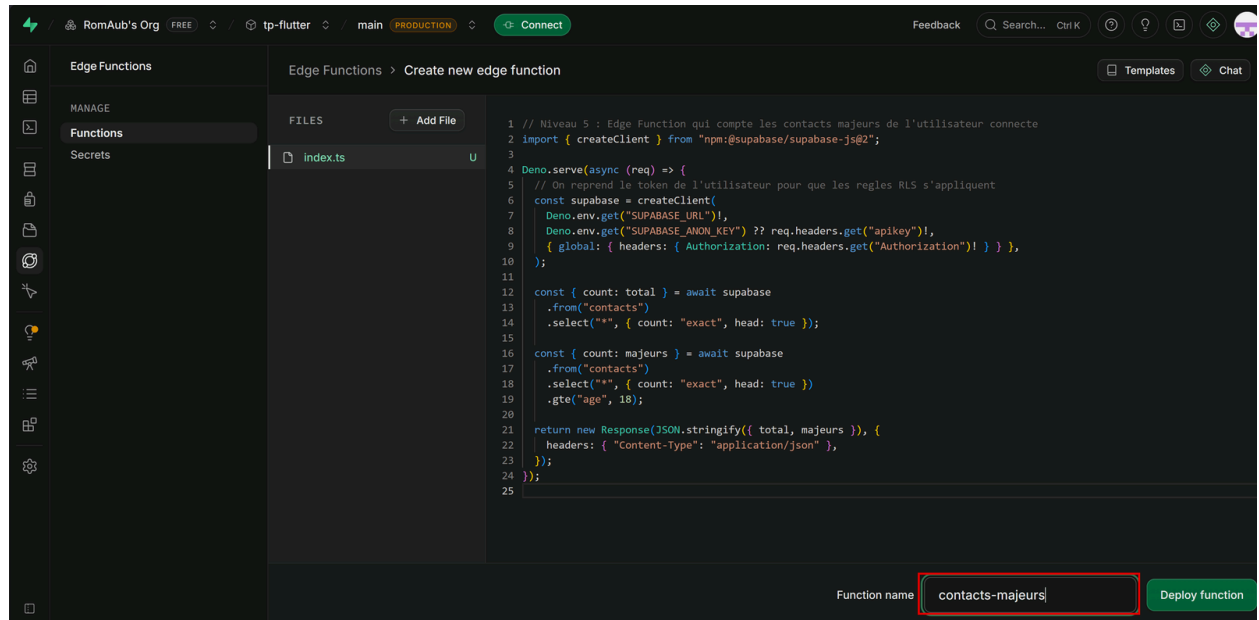
  String nomDuGroupe(dynamic id) {
    for (final g in groupes) {
      if (g['id'] == id) return g['nom'];
    }
    return 'Sans groupe';
  }

  // CREATE
  Future<void> ajouterContact(
```

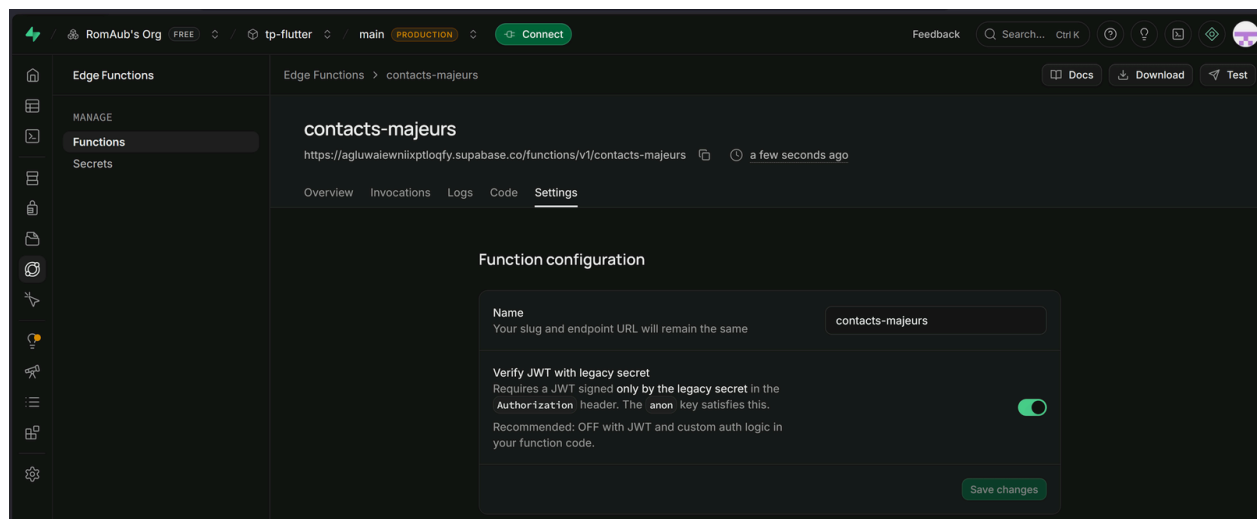
On a pas pu tester avec deux appareils en même temps car le PC n'a pas assez de mémoire pour lancer deux émulateurs.

Niveau 5 : Edge Function

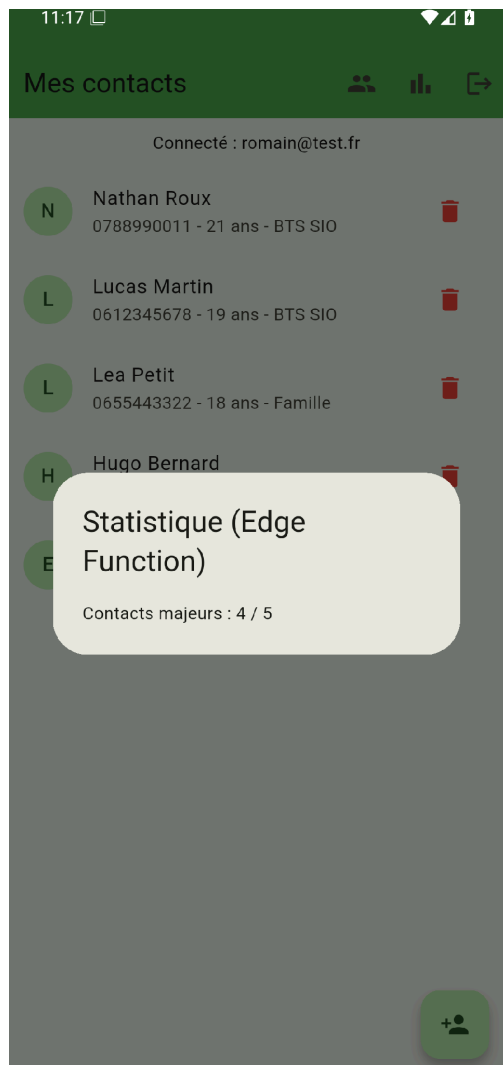
On crée une **Edge Function** “contacts-majeurs” directement depuis la console (pas besoin de la CLI) qui compte les contacts majeurs de l'utilisateur connecté. Elle reprend le token de l'utilisateur donc la RLS s'applique aussi :



On la déploie :



Elle est appelée depuis l'application avec `supabase.functions.invoke()` (bouton statistique en haut) :



Comparatif Firebase / Supabase

Critère	Firebase	Supabase
Base de données	NoSQL (collections / documents)	PostgreSQL (tables, SQL)
Relations	Pas de jointure, il faut dupliquer ou faire plusieurs requêtes	Clés étrangères et jointures dans le select()
Sécurité	Règles Firestore (langage spécial)	Row Level Security en SQL
Temps réel	snapshots()	stream()
Stockage de fichiers	Payant (forfait Blaze)	Inclus dans le forfait gratuit
Configuration Flutter	google-services.json + Gradle	URL + clé dans le code
Licence	Propriétaire (Google)	Open source, auto-hébergeable

Pour moi **Supabase** est plus simple à configurer (juste une URL et une clé, pas de fichier Gradle à modifier) et on réutilise ce qu'on connaît déjà en **SQL**. **Firebase** est plus mature et très bien intégré avec Flutter mais le fait que Storage soit devenu payant est un vrai problème pour un projet étudiant.

Conclusion

Ce TP nous a permis de configurer un projet Firebase dans une application Flutter, de manipuler **Cloud Firestore** (ajout, lecture en temps réel, modification et suppression) et de gérer l'inscription / connexion avec **Firebase Authentication**.

Comme Firebase Storage n'est plus gratuit, on a refait la photo de profil avec **Supabase** et fait les exercices du cours, ce qui nous a fait découvrir la **RLS**, les jointures et les **Edge Functions**. Les tutoriels étaient assez anciens, il a fallu adapter pas mal de choses (Gradle en Kotlin, menus de la console, versions des packages).